

SensorChat: Answering Qualitative and Quantitative Questions during Long-term Multimodal Sensor Interactions

XIAOFAN YU, University of California San Diego, USA

LANXIANG HU, University of California San Diego, USA

BENJAMIN REICHMAN, Georgia Institute of Technology, USA

DYLAN CHU, University of California San Diego, USA

RUSHIL CHANDRUPATLA, University of California San Diego, USA

XIYUAN ZHANG, University of California San Diego, USA

LARRY HECK, Georgia Institute of Technology, USA

TAJANA S ROSING, University of California San Diego, USA

Natural language interaction with sensing systems is crucial for addressing users' personal concerns and providing health-related insights into their daily lives. When a user asks a question, the system automatically analyzes the full history of sensor data, extracts relevant information, and generates an appropriate response. However, existing systems are limited to short-duration (e.g., one minute) or low-frequency (e.g., daily step count) sensor data. In addition, they struggle with quantitative questions that require precise numerical answers.

In this work, we introduce SensorChat, the first end-to-end QA system designed for daily life monitoring using long-duration, high-frequency time series data. Given raw sensor signals spanning multiple days and a user-defined natural language question, SensorChat generates semantically meaningful responses that directly address users' concerns. SensorChat effectively handles both quantitative questions that require numerical precision and qualitative questions that require high-level reasoning to infer subjective insights. To achieve this, SensorChat uses an innovative three-stage pipeline that includes question decomposition, sensor data query, and answer assembly. The first and third stages leverage Large Language Models (LLMs) to interpret human queries and generate responses. The intermediate querying stage extracts relevant information from the complete sensor data history, which is then combined with the original query in the final stage to produce accurate and meaningful answers. Real-world implementations demonstrate SensorChat's capability for real-time interactions on a cloud server while also being able to run entirely on an edge platform after quantization. Comprehensive QA evaluations show that SensorChat achieves 93% higher answer accuracy than the best performing state-of-the-art systems on quantitative questions. Furthermore, a user study with eight volunteers highlights SensorChat's effectiveness in answering qualitative and open-ended questions. The code is available at <https://github.com/Orienfish/SensorChat>.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; • **Computing methodologies** → **Machine learning**; *Natural language processing*.

Additional Key Words and Phrases: Question Answering, Multimodal Sensors, Large Language Models

Authors' Contact Information: Xiaofan Yu, University of California San Diego, La Jolla, California, USA, x1yu@ucsd.edu; Lanxiang Hu, University of California San Diego, La Jolla, California, USA, lah003@ucsd.edu; Benjamin Reichman, Georgia Institute of Technology, Atlanta, Georgia, USA, bzt@gatech.edu; Dylan Chu, University of California San Diego, La Jolla, California, USA, dchu@ucsd.edu; Rushil Chandrupatla, University of California San Diego, La Jolla, California, USA, ruchandrupatla@ucsd.edu; Xiyuan Zhang, University of California San Diego, La Jolla, California, USA, xiyuanzh@ucsd.edu; Larry Heck, Georgia Institute of Technology, Atlanta, Georgia, USA, larryheck@gatech.edu; Tajana S Rosing, University of California San Diego, La Jolla, California, USA, tajana@ucsd.edu.



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

© 2025 Copyright held by the owner/author(s).

ACM 2474-9567/2025/9-ART148

<https://doi.org/10.1145/3749496>

ACM Reference Format:

Xiaofan Yu, Lanxiang Hu, Benjamin Reichman, Dylan Chu, Rushil Chandrupatla, Xiyuan Zhang, Larry Heck, and Tajana S Rosing. 2025. SensorChat: Answering Qualitative and Quantitative Questions during Long-term Multimodal Sensor Interactions. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 9, 3, Article 148 (September 2025), 35 pages. <https://doi.org/10.1145/3749496>

1 INTRODUCTION

In recent years, the rapid growth of Internet of Things (IoT) devices has made them a ubiquitous part of daily life, from smartphones and smart home accessories to wearable technologies [13]. These devices, equipped with multimodal on-device sensors, enable continuous monitoring of human activities and environmental conditions. The large volumes of long-term sensor data collected by these devices can provide a comprehensive view of users' daily routines and valuable insights into their health. While traditional research has focused on building classification models for tasks like human activity recognition [10, 41, 43, 54, 58, 59], recent work has highlighted the value of natural language interfaces for making sensor data more accessible and useful to humans [9, 11, 27, 40, 55, 60]. Question Answering (QA) systems, where users ask questions and receive responses from the sensor systems, offer a flexible and intuitive way to engage with sensor data. Unlike passive lifelogs or visualizations [6, 7, 26, 49, 57], QA allows users to actively express their interests and needs, making the system more responsive to their personalized concerns.

A useful QA system should be able to handle long-term sensor data and a wide range of practical queries. We envision users collecting sensor data through everyday mobile devices (like smartphones and smartwatches) equipped with multimodal sensors, such as IMUs, microphones, GPS, and compasses. These sensors operate continuously in the background, capturing rich streams of data that can be stored locally or sent to the cloud. When users have specific questions or want to seek insights, they interact with the QA system, which should automatically understand their intent, analyze historical raw sensor data to extract relevant information, and generate meaningful answers. The QA system can run on mobile devices or in the cloud, depending on resource needs and privacy requirements. To build a truly practical QA system, we focus on *long-duration* sensor data that spans more than a day, enabling the analysis of extended or recurring behavioral patterns (e.g., daily routines or lifestyle trends) that are not observable in shorter time frames. In addition, the system should be capable of processing *high-frequency* raw time series data to capture fine-grained activities, like gym workouts, which cannot be detected through sparse sensor readings such as daily step counts. On the query side, existing research for monitoring daily life has mainly focused on two broad categories: *quantitative* and *qualitative* questions [22, 47, 55]. Quantitative questions require precise, objective answers derived from sensor data [47, 55], often involving content such as true/false, location, activity, count, or time duration (e.g., “How long did I exercise yesterday?”). In contrast, qualitative questions involve interpreting sensor data to infer high-level, subjective insights [22]. Qualitative questions can be vague, open-ended and often require external knowledge. For example, answering “Did I exercise enough for a young adult?” requires understanding of the standard activity levels. We recognize that addressing both categories of questions is important in building a useful QA system. To clarify our problem scope, we summarize the key concepts in Table 1.

In this paper, we explore the core research question: *How can we design a practical QA system that delivers accurate and meaningful answers to both quantitative and qualitative user queries, based on long-duration, high-frequency sensor data?* We recognize three key challenges for this problem:

- (1) **Processing long-duration and high-frequency sensor data** is inherently challenging due to the massive volume of raw data generated. For instance, a continuously running IMU sensor with 9 axes at a 40Hz sampling rate can produce roughly 125MB of data per day. In practice, mobile devices are equipped with a wide array of sensors beyond IMUs, further increasing the complexity and size of the data. Extracting

Table 1. Key concepts that defines the problem scope of this paper. These concepts are motivated by practical user needs.

Concept	Meaning
Long-duration data	Sensor data spans more than a full day of user activity
High-frequency data	Collected from high-frequency sensors (e.g., IMU raw signals), as opposed to sparse data like daily step counts or statistics (e.g., mean value)
Quantitative questions	Require precise and objective answers based on sensor data, such as true/false, location, activity, count, or time duration
Qualitative questions	Require high-level reasoning to infer subjective insights, such as work-life balance and social interactions, possibly involving external knowledge

relevant information from such vast, multimodal time series data becomes a difficult task that involves pattern recognition and temporal query retrieval.

- (2) **Effectively fusing variable duration of sensor data with textual questions** poses a significant challenge to answer certain questions. For instance, a specific quantitative question like "*What was I doing at 10:00 AM yesterday?*" requires the system to precisely locate and interpret sensor data from a specific timestamp. In contrast, a high-level qualitative question such as "*How would you rate my lifestyle?*" demands a broader understanding of patterns across extended periods. Effectively addressing these diverse queries requires flexible reasoning across multiple time scales, along with the ability to align temporal sensor data with natural language queries.
- (3) **Generating accurate and semantically meaningful answers** makes QA fundamentally more challenging than traditional classification or journaling tasks. In contrast to conventional human activity recognition [10, 41, 43, 54, 58, 59], which typically involves selecting from 10–20 predefined activity classes, QA requires choosing the next appropriate token from a vocabulary of over 32,000 possibilities at each step [50]. Unlike passive journaling systems [6, 7, 26, 49, 57] that simply log events, QA systems need to understand the user's intent and deliver relevant, satisfying responses. The response needs to be a numerically precise answer to a quantitative question or a thoughtful, semantically rich interpretation for a qualitative question. This high level of complexity demands deeper reasoning, language understanding, and tight alignment between sensor data and user queries.

Existing research falls short in addressing all above challenges. Current QA systems are limited to short sensor durations [9, 11, 21, 39, 55] (e.g., one minute, as shown in Fig. 1a) or low-frequency readings [17, 30, 60] (e.g., daily step counts and temperature, as shown in Fig. 1b). In Section 3.2, we demonstrate that directly applying these systems to long-duration, high-frequency sensor data results in poor answer quality due to the increased data volume and complexity. Although recent advances in Large Language Models (LLMs) and multimodal LLMs offer new ways to understand raw sensor data, current approaches are ineffective for handling long-duration, high-frequency sensor data in our setting. These approaches typically involve converting raw sensor signals into text [23, 28, 34, 56], visualizing them as images [62], or encoding them into embeddings before feeding them into an LLM [11, 21, 37, 38]. However, as detailed in Appendix A, uncompressed text or embeddings often exceed the token limit of LLMs (e.g., 8,092 tokens for GPT-4 [5]), while compressing high-frequency sensor readings over a day into a single image results in loss of interpretability. To the best of our knowledge, no existing system effectively addresses all three challenges in our setting.

In this paper, we introduce SensorChat, the *first* end-to-end system designed for answering both qualitative and quantitative queries in long-term sensor monitoring, using multimodal and high-frequency time series sensors. In contrast to prior QA systems [9, 11, 17, 21, 30, 39, 55, 60], SensorChat is capable of processing long-duration and high-frequency sensor data spanning weeks or even months. SensorChat generates accurate answers to quantitative questions and semantically meaningful answers to qualitative questions, all based on the extracted sensor information. Figure 1c illustrates two QA examples using SensorChat. The input consists of raw multimodal

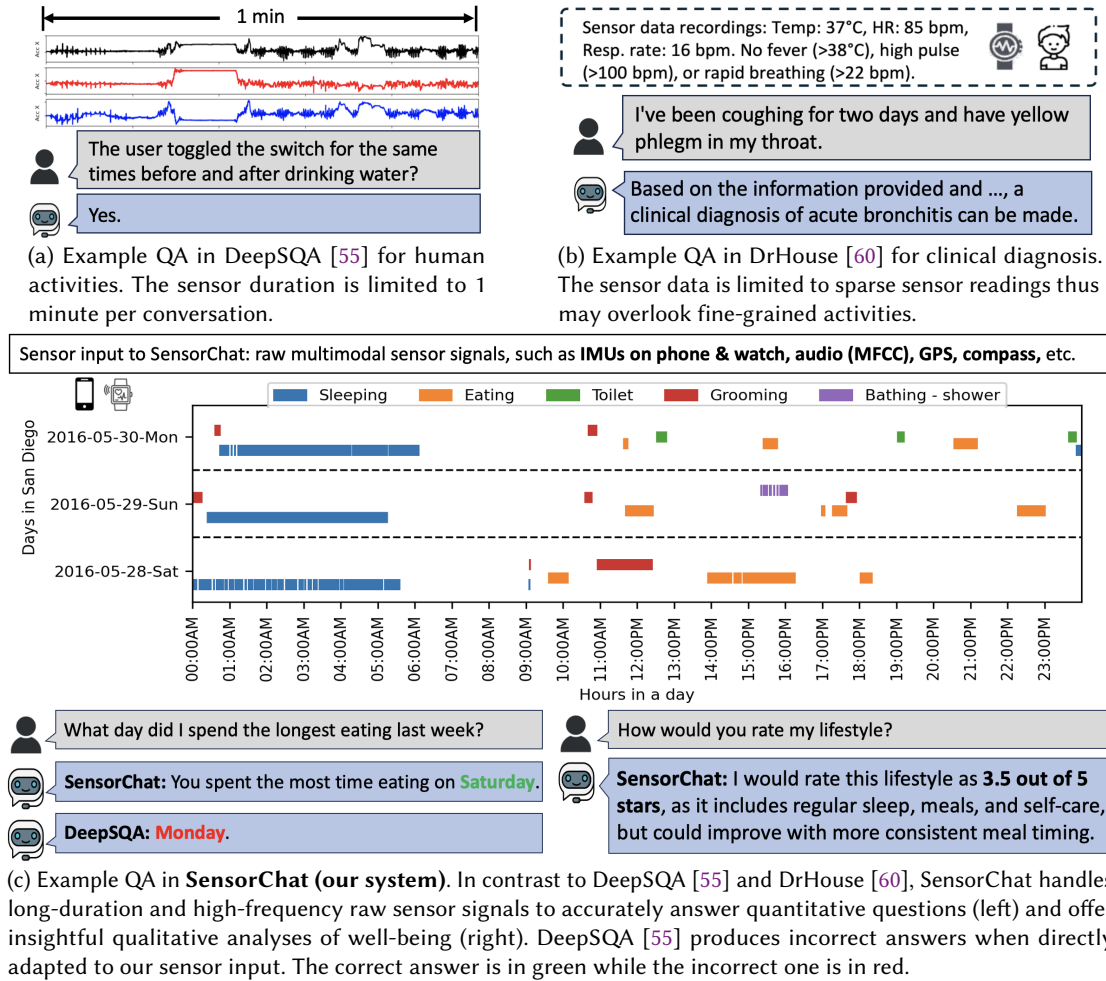


Fig. 1. A visual comparison existing QA systems for sensor data and SensorChat.

sensor data collected over three days from a smartphone and a smartwatch. For clarity, we visualize the inferred user activity labels, with the x-axis showing time of day (00:00 AM to 11:59 PM) and the y-axis indicating separate days. Different activities are represented by color-coded bars. When asked a quantitative query like “*What day did I spend the longest eating last week?*”, SensorChat automatically understands the question and searches the full sensor history to return a precise answer. SensorChat can also handle open-ended qualitative questions, such as queries “*How would you rate my lifestyle?*” by reasoning on top of a detailed analysis of daily routines.

To achieve the goal, SensorChat introduces a novel three-stage design including question decomposition, sensor data query and answer assembly. The key design philosophy of SensorChat is to use LLMs in the first and third stages to correctly understand user questions and generate natural language answers, while inserting an explicit query stage in between for accurate sensor data processing and effective sensor-question fusion. SensorChat tackles the first challenge of processing long-duration, high-frequency sensor data by pretraining a sensor encoder and a text encoder using a novel contrastive loss. This pretraining enables the model to associate raw sensor signals with semantically meaningful text embeddings. Then, in the second stage, SensorChat uses

a custom query procedure to identify relevant sensor segments from the full monitoring history based on the textual input, bridging sensor data and natural language. Lastly, to ensure high-quality responses, SensorChat uses in-context learning [48] and Chain-of-Thought (CoT) prompting [15] during question decomposition to correctly interpret user intent. In the answer assembly stage, SensorChat applies Low Rank Adaptation (LoRA) finetuning [24] to align answers with user preferences, based on a state-of-the-art dataset [47]. The three stages of SensorChat work collaboratively to deliver accurate and meaningful responses to both quantitative and qualitative questions over long-term, high-frequency multimodal sensor data.

We implement SensorChat in two system variants to accommodate various user needs: SensorChat_C for cloud deployment and SensorChat_E for edge deployment. SensorChat_C uses OpenAI's GPT APIs [5, 61] to interpret user queries, offering real-time QA interactions on a server with an NVIDIA A100 GPU [3]. On the other hand, SensorChat_E uses quantized LLMs and runs entirely on a representative edge device of NVIDIA Jetson Orin NX [2]. SensorChat_E preserves user privacy albeit at the cost of higher latency. We evaluate SensorChat through two sets of experiments to test different capabilities of the system. First, we test SensorChat on SensorQA [47], a state-of-the-art QA dataset focused on quantitative questions. Next, we conduct a real-world user study with eight volunteers, approved by the Institutional Review Board (IRB), to assess SensorChat's performance on both qualitative and quantitative questions. This study also tests SensorChat's generalizability to raw sensor data collected by personal mobile devices.

In summary, the major contributions of our work are:

- We propose SensorChat, the *first* end-to-end system for answering qualitative and quantitative questions in long-term monitoring using high-frequency time series sensors. SensorChat is designed with a novel three stage pipeline, including question decomposition, sensor data query and answer assembly.
- We introduce a novel contrastive sensor-text pretraining loss for training the sensor and text encoders offline. We also design a custom query procedure to ensure accurate sensor embedding queries given arbitrary text during the online question-answering.
- We enhance the LLMs in SensorChat through in-context learning, CoT and fine-tuning, incorporating carefully designed prompts to improve the quality of question decomposition and answer assembly.
- Evaluations on a quantitative QA dataset show that SensorChat improves answer accuracy by 93% compared to the best performing state-of-the-art QA systems. A real-world user study confirms that SensorChat provides meaningful and satisfying answers to users' questions, including both quantitative and qualitative ones, while generalizing to personal mobile devices.
- We implement SensorChat in two system variants, SensorChat_C and SensorChat_E. SensorChat_C enables real-time user interactions from a cloud server, while SensorChat_E operates fully on local edge devices with reasonable answer latency.

2 RELATED WORK

Question Answering using Sensor Data In the sensing domain, prior QA systems can be broadly categorized based on the type and duration of sensor data used, as summarized in Table 2. One line of research focuses on understanding users' current activities using real-time sensor readings, as shown in Fig. 1a. For each QA instance, DeepSQA [55] processes one-minute IMU signals, Sensor2Text [11] uses 16-second body tracking data, PrISM-Q&A [9] analyzes short-segment smartwatch signals for procedural guidance via a voice assistant, OneLLM [21] allows 10-second IMU signals as one of the eight modalities. All of these works are limited to short-duration sensor data, whereas directly applying these methods to long-duration sensor data in our setting leads to ineffective fusion between sensor and text, as we detail in Sec. 3.2. On the contrary, SensorChat supports understanding and reasoning over long-term data spanning more than a day. The second line of research uses long-term but sparse sensor data to support health-related queries and clinical assessments. Enghardt *et al.* [17],

Table 2. Comparison of SensorChat with existing sensor-based QA systems. SensorChat is the first QA system to provide accurate, meaningful answers based on long-duration, high-frequency time series sensor data.

Existing QA systems	Long-duration sensor data	High-frequency time series sensors
DeepSQA [55], Sensor2Text [11], PrISM-Q&A [9], OneLLM [21]	×	✓
Englhardt <i>et al.</i> [17], Health-LLM [30], DrHouse [60]	✓	×
SensorChat (this work)	✓	✓

Health-LLM [30], and DrHouse [60] converted physiological signals (e.g., heart rate, step counts) into text prompts for LLMs, as exemplified in Fig. 1b. However, these methods rely on low-frequency data, thus cannot capture fine-grained activities like gym sessions. In contrast, SensorChat processes raw, high-frequency sensor signals, capturing detailed user schedules and activity durations.

Multimodal LLMs using Sensor Data The rise of LLMs has sparked growing interest in integrating sensor data into LLMs for high-level reasoning. HARGPT [28], Penetrative AI [56], Hota *et al.* [23], and Liu *et al.* [34] are among the first works to convert raw sensor signals into text prompts for LLMs. By-My-Eyes [62] selects visualization tools to present sensor data as visual prompts. Other efforts in multimodal LLMs [21, 38] train adapters to convert raw sensor signals into embeddings, which are then used for finetuning LLMs. However, these methods struggle when tasked with interpreting the long-duration, high-frequency data in our scenario, as discussed in Appendix A. More recent approaches like IMU2CLIP [39] and LLMsense [42] use classifiers to map raw signals into narrative logs for LLM input, but still suffer from LLMs’ limitations in handling long context [20, 31], i.e., input with more than 2K tokens. In contrast, SensorChat overcomes this limitation by incorporating a dedicated sensor data query stage, enabling it to handle long-term sensor queries effectively.

3 BACKGROUND AND MOTIVATION

In this section, we formally define the problem we address and present a motivating study highlighting the limitations of state-of-the-art QA techniques.

3.1 Problem Statement

We formulate the problem as generating an appropriate answer based on a given question and sensor input. In the i th interaction round, the input and output are a pair of natural language question and answer, denoted as (q^i, a^i) for $i \in 1, \dots, N$, where N is the total number of QA interactions. The sensor input consists of the full history of raw sensor signals $\mathbf{x} \in \mathbb{R}^{T \times d}$, where T is the number of recorded time steps and d is the number of multimodal sensors. We specifically target long-duration, high-frequency sensor data (e.g., raw IMU signals sampled at 40Hz over more than a day), resulting in extremely large T . Formally, our objective is formed as training an autoregressive model, ω , to predict the next token in the answer sequence:

$$\max_{\omega} \sum_{i=1}^N \sum_{j=1}^m \log p_{\omega} \left(a_j^i \mid \mathbf{x}, q^i, a_{1:j-1}^i \right), \quad (1)$$

where i iterates through all QA pairs, j iterates through all tokens in the answer and m is the maximal length for answers. Given a QA dataset (q^i, a^i) , ω is trained to maximize the likelihood of the correct next token. During inference, ω is fixed, and the model selects the next token with the highest predicted probability.

Question and Answer Types We target at two broad categories of user queries q^i : quantitative and qualitative questions, as commonly identified in prior work [22, 47, 55]. For quantitative questions, we mainly focus on the six question types and seven answer types defined by the SensorQA dataset [47], a state-of-the-art quantitative QA benchmark for daily-life monitoring. Table 3 shows the detailed question and answer types. SensorQA is

Question Categories	Example Questions	Answer Categories	Example Answers
Time Compare	Did I spend more time sitting or standing?	Action	Doing computer work
Day Query	On which day did I spend the most time at home?	Day/Days	Last Friday
Time Query	How long was I in class and at school?	Existence	Yes/No
Counting	How often did I groom?	Time Length	40 Minutes
Existence	Did I have a meeting on Wednesday?	Location	At school
Action Query	What did I do after I left home on Tuesday?	Count	Three times
		Timestamp	Around 11:00 am

(a) Question categories.

(b) Answer categories.

Table 3. Quantitative Q&A categories that we focus on in this paper, as identified in the SensorQA dataset [47]. The shorted version of the answers are presented for simplicity.

chosen for its realism, as its questions were written by human annotators simulating interactions with a smart sensor system. Due to the lack of a benchmark for qualitative QA using long-term sensor data, we focus on general topics that affect quality of life, such as work–life balance and social interactions [22]. We evaluate SensorChat on qualitative questions through a real user study in Sec. 7.

3.2 Limitations of State-of-the-Art (SOTA) Methods

With a concrete problem setup, we conduct a motivating study to understand the limitations of state-of-the-art systems on this problem. We then discuss the takeaways which inspire the design of SensorChat.

Study Setup We select the SensorQA dataset¹ [47] because it has the best alignment with QA scenarios using long-duration, high-frequency multimodal sensor data. SensorQA has raw sensor measurements collected using smartphone and smartwatch sensors, including IMUs, audio (MFCC features), and phone state sensors (compass, GPS location, Wi-Fi, light, etc.). It contains data from 60 subjects and has more than 300K minutes of sensor recordings, annotated with 51 context labels after cleaning. In total, SensorQA [47] contains 5.6K quantitative QA pairs reflecting practical human interests in daily life monitoring, including approximately 3K questions on a single day and 2.6K for longer durations spanning weeks or months. We randomly split the data in SensorQA [47] into an 80/20 train-test set. Raw sensor data is split into time windows of 20 seconds for models to process.

We train and evaluate three methods representing SOTA solutions using neural network and LLM-based models: (1) **LLaMA2-7B** [50], as a reference, is the LLM baseline that considers only the question without sensor data. (2) **DeepSQA** [55] represents the SOTA CNN-LSTM based model for fusing the question and time series sensor data. While DeepSQA was originally designed for short-duration sensor data, we adapt it by using the full history of IMU signals as sensor-side input. (3) **LLMSense** [42] is the SOTA LLM-based method for reasoning over sensor data. It converts neural network classifications into a narrative log including all timestamps, and then feeds this log and the user question into an LLM to generate an answer. Detailed implementations of these methods are discussed in Appendix D. We focus on answer accuracy for quantitative questions in SensorQA. An answer is considered correct if it contains the key phrases from the ground-truth answer provided by SensorQA. Otherwise, it is marked as incorrect. For example, in Fig. 1c, if the ground-truth answer is “*You spent the longest eating time on Saturday*,” then an answer that includes “*Saturday*” is correct. An answer like “*Monday*” is incorrect.

Fig. 2 shows the accuracy results of SOTA methods by question category (left) and answer category (right). The key observation is that all SOTA baselines deliver low answer accuracy. LLaMA2-7B [50], DeepSQA [55], and LLMSense [42] achieve average accuracies of 0.26, 0.27, and 0.23, respectively. Notably, DeepSQA and LLMSense—both of which attempt to integrate sensor data with the question—perform comparably to or worse than LLaMA2-7B, which does not incorporate any sensor input. This suggests that existing approaches struggle to handle the complexity introduced by long-duration, high-frequency sensor data. In particular, DeepSQA’s

¹The dataset is available at <https://github.com/benjamin-reichman/SensorQA>.

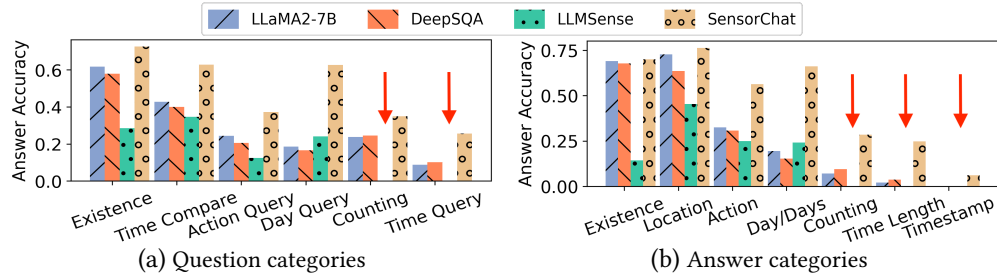


Fig. 2. Answer accuracy of state-of-the-art methods and SensorChat on SensorQA [47]. Categories are arranged from easier questions/answers on the left to more challenging questions/answers on the right. Red arrows highlight the question/answer types that existing works struggle with.

performance suffers when directly fed with more than a day’s worth of IMU signals. The increased input size and complexity significantly complicate the fusion process, overwhelming its relatively simple model architecture. On the other end, LLMsense performs poorly on questions requiring precise numerical reasoning, such as counting or time queries (highlighted by red arrows in Figure 2). This is due to LLMsense’s reliance on LLMs to reason over long narrative logs, which contains up to 1,440 lines for a single day. Prior studies have shown that LLMs often struggle with long-context numerical tasks and are prone to hallucinations in such settings [20, 25, 31].

In summary, directly applying existing QA methods to long-duration, high-frequency sensor data leads to poor performance, particularly in quantitative reasoning tasks. These limitations highlight the need for a new system capable of effectively processing raw sensor inputs at scale and fusing them with practical user queries. To address this, we propose SensorChat, a novel system with a three-stage pipeline that includes a dedicated sensor data query stage. As shown in Fig. 2, SensorChat significantly improves answer accuracy across all categories, particularly for quantitative tasks such as day comparisons and time queries.

4 SensorChat DESIGN

4.1 Overview of SensorChat

We design SensorChat, a novel end-to-end QA system for answering both quantitative and qualitative questions using long-duration, high-frequency raw sensor data. SensorChat achieves the goal by having a three-stage pipeline: *question decomposition*, *sensor data query*, and *answer assembly*. SensorChat addresses all three challenges discussed in Sec. 1, i.e., processing long-duration, high-frequency sensor data, effectively fusing sensor data with text questions, and generating accurate, semantically meaningful answers. The sensor query stage is the *key novel component* of SensorChat. In the query stage, SensorChat pretrains a sensor encoder and a text encoder offline to effectively associate high-frequency and long-duration multimodal sensor timeseries with semantically meaningful embeddings. To effectively fuse sensor and text modalities, SensorChat further designs a custom online query mechanism using a similarity search, retrieving all sensor information relevant to the original question. This enables precise extraction of sensor context across varying durations and question types, significantly improving the accuracy of the final answer. Last but not least, SensorChat utilizes two LLMs: one in the question decomposition stage to interpret user queries, and the other in the answer assembly stage to generate the final response, further enhanced through in-context learning and fine-tuning techniques.

The complete pipeline of SensorChat is illustrated in Fig. 3. For example, given a question like, “How long did I exercise last week in the morning?” (① in Fig. 3), SensorChat first decomposes the question and generates specific sensor data queries using LLMs. These queries include details such as the context of interest, date and time ranges, and the query function (③). In this case, the query might specify a context of “do exercise,” a time span of “last week,” a time of day of “morning,” and a query function of CalculateDuration. To improve

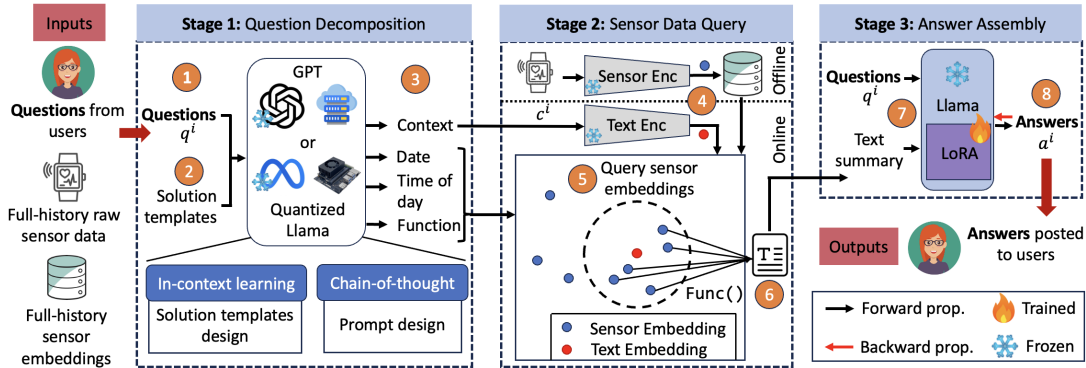


Fig. 3. The system diagram of SensorChat including three stages. When a user asks a question, SensorChat first decomposes the question (stage 1), queries the relevant sensor embeddings (stage 2), and generates accurate, semantically meaningful answers (stage 3). The sensor and text encoder used in the query stage are pretrained offline in advance.

the accuracy of question decomposition, SensorChat uses solution templates for in-context learning (②) with carefully designed prompts. In-context learning is chosen because it enhances the quality of LLM outputs using only a small number of solution templates, compared to finetuning [19]. Next, the sensor query stage encodes the context label “exercise” into a text embedding (④) and retrieves sensor embeddings that are sufficiently similar to the text embedding (⑤). These sensor embeddings are encoded offline from the full-history raw sensor data. Both the sensor and text encoders are pretrained offline to align their outputs in the same embedding space, ensuring accurate sensor data retrieval. Therefore, only text encoding and similarity-based query searches are performed online. Additional properties, such as the date (“last week”) and the time of day (“morning”), are used to constrain the query range. The final step of the sensor data query stage involves summarizing the relevant sensor embeddings into text using the corresponding query function (⑥). For instance, with the `CalculatedDuration` function, this step calculates the total duration of the retrieved sensor embeddings. This results in a sensor context such as: “Among all days last week, you exercised for 35 minutes on Monday morning and 55 minutes on Thursday morning.” Finally, the original question and the sensor context are sent to the answer assembly stage, where a fine-tuned LLM generates the final answer (⑦). Finetuning is used to align the model’s output with human-generated answers from state-of-the-art datasets. With accurate sensor context, the model generates a precise response, such as: “You exercised a total of 1 hour and 30 minutes last week.” (⑧). In the following lines, we describe the detailed designs in each stage of SensorChat: question decomposition (Sec. 4.2), sensor data query (Sec. 4.3) and answer assembly (Sec. 4.4).

4.2 Question Decomposition

The question decomposition stage processes the user question and identifies specific query triggers for the sensor data query stage, such as the context of interest, date and time ranges, and the query function, as illustrated in the leftmost box in Fig. 3. We choose to rely on LLMs for this stage due to their outstanding capability in handling natural language inputs. However, LLMs are not without limitations as they can produce erroneous outputs or hallucinations [25]. The key challenge in designing this stage lies in generating *accurate* decompositions for *arbitrary* user inputs, particularly in the absence of a dedicated dataset for this task. To address this, SensorChat leverages the capabilities of a pre-trained LLM by employing in-context learning [48] and chain-of-thought techniques [15]. These approaches are chosen because, compared to finetuning [24], they enhance the quality of LLM outputs using a small number of manually crafted examples, eliminating the need for a high-quality, task-specific dataset.

"You are a helpful assistant tasked with querying sensor data from a database to answer user questions. Generate a query to extract the relevant sensor data using specific code functions. Make sure to highlight the code function name with <<>>, highlight the corresponding activity with (()), highlight the corresponding date with [[]] and the time of day with |||. You can call from {List of functions}. Below are some examples.\n {Solution templates}\n\n Now please generate the solution and **step-by-step** explanation for the following question.\n Question: {Question}"

(a) The prompt design for the question decomposition stage includes ICL and CoT. A list of available query functions, solution templates, and the user's question need to be included in the prompt.

"Question": "How much total time did I spend eating on Thursday morning?",
 "Solution": "Query the database with the function <<calculate_duration>> using the activity ((eating)), the date [[Thursday]] and the time of day ||morning||.",
 "Explanation": "The question asks for the total duration, thus we can query the database with the function <<calculate_duration>> to obtain the total duration. The question specifically asks for the total duration of ((eating)), thus we can query using the activity ((eating)). The question also asks for the total duration on [[Thursday]], thus we can query the date [[Thursday]]. Finally, the question specifically focuses on the time ||morning||, thus we can query the database using the time of day ||morning||."

(b) Example solution template for a time-related query to be inserted into the prompt to question decomposition.

Fig. 4. Prompts design in the question decomposition stage of SensorChat to enhance the generating quality of LLMs.

An example prompt for question decomposition is illustrated in Fig. 4a. We instruct the LLM to mark different arguments with distinct symbols, such as, using "<<" and ">>" for function names, "(" and ")" for context labels, etc. This enables accurate extraction of multiple arguments from a single LLM interaction. SensorChat also supports multiple extracted phrases to handle complex questions such as "How long did I work at school on Monday and Tuesday?" The question decomposition stage is designed with the flexibility to support new extractable properties. We next explain more details on the in-context learning and chain-of-thought techniques.

In-Context Learning (ICL) integrates a few examples directly into the prompt during inference, enabling LLMs to adapt effectively to specific tasks without requiring fine-tuning [8, 48]. Building on this insight, we design a library of general solution templates covering diverse QA scenarios and incorporate them as in-context examples to improve decomposition accuracy (Fig. 4a). We observe that decomposition solutions for the same question type often share similarities. For instance, "how long" questions typically map to the CalculateDuration function. To utilize this question-specific property, in SensorChat, we first classify the question type using a BERT model [16]. Then, we dynamically select templates that best match the question category. This strategy has been shown to improve ICL performance, even in settings that are different from ours but face the same challenge of limited annotated data [19]. Fig. 4b illustrates an example of a solution template for a time query.

Chain-of-Thought (CoT) Prompting explicitly requests LLMs to generate their reasoning process, which improves accuracy in zero- or few-shot logical reasoning [15, 36]. In SensorChat, we include "please generate step-by-step explanations" in prompts to encourage CoT, as shown in Fig. 4a. CoT and detailed solution templates enhance logical reasoning, especially for multi-step questions.

4.3 Sensor Data Query

The sensor query stage, as the most important module in SensorChat, is responsible for processing the long-duration, high-frequency raw sensor signals and extracting the most relevant information given arbitrary user queries. This enables effective fusion of sensor data with natural language. As shown in the middle part of Fig. 3, the sensor data query stage takes the extracted arguments from question decomposition, such as the context of interest (e.g., "do exercise"), time span (e.g., "last week"), time of day (e.g., "morning") and query function (e.g., CalculateDuration). The output of the stage is a concise summary of the relevant sensor data in text form. SensorChat introduces two novel components to addresses these challenges. First, during offline training, SensorChat pretrains a sensor encoder and a text encoder using a novel contrastive loss to align sensor signals

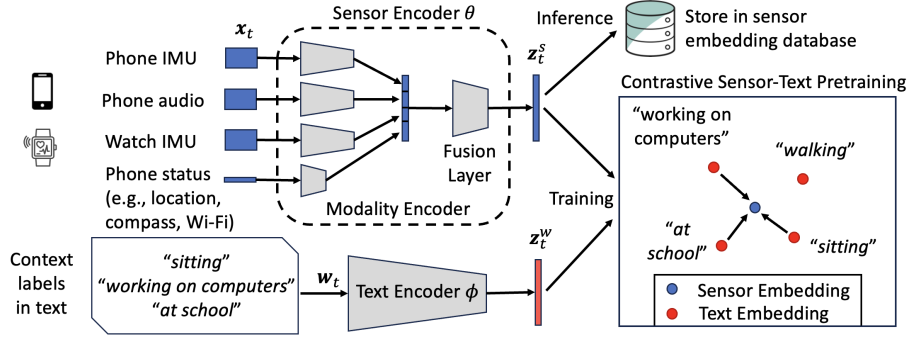


Fig. 5. Visualization of the contrastive sensor-text pretraining in SensorChat on a sample $\{x_t, w_t\}$. At time t , the raw sensor input x_t includes data from phone and watch IMUs, audio, and phone status. The time series signals are split into time windows. The positive context labels are $w_t = \{\text{sitting, working on computers, at school}\}$ while other labels such as “walking” are negative ones. Our pretraining loss increases the similarity between x_t and the positive labels, and reduces it for the negative ones.

with partial context (Sec. 4.3.1). Then, during online querying, SensorChat designs a set of custom query functions to extract relevant sensor embeddings, which are converted to textual summarizations for the final answer assembly stage (Sec. 4.3.2).

4.3.1 Contrastive Sensor-text Pretraining for Partial Contexts. The sensor and text encoders are crucial for effective sensor-text fusion, serving as a “bridge” between high-frequency sensor timeseries and semantic text. A key challenge in SensorChat is ensuring that the sensor and text encoders can handle diverse context phrases extracted from user questions, which may vary unpredictably based on users’ habits. Suppose c_i represents the context argument extracted from question q_i . Since SensorChat is designed for general daily life monitoring, c_i can refer to anything from activities, locations, or social interactions (e.g., spending time with family), depending on the user’s interest. For example, in the question “Did I go to school?”, c_i is “go to school,” which refers to a location. This means the encoders in SensorChat must be able to align sensor data with a broad range of contexts. However, achieving this level of alignment is difficult. Existing pretraining methods like CLIP and its variants [39, 45] work well when aligning sensor data with full, detailed sentences (e.g., “The person is sitting and working on computers at school”). However, when a user is specifically interested in a partial context like “working on computers,” the encoded text embedding may not closely match the original sensor embeddings, leading to reduced accuracy in sensor data query as we show in Appendix F. To address this challenge, we introduce a novel contrastive sensor-text pretraining loss for partial contexts.

Fig. 5 presents the details of our pretraining process. We pretrain our model on a large-scale multimodal sensor dataset with annotations, where each sample $\{x_t, w_t\}$ consists of one segment of raw time series sensor data x_t collected at time t and an associated set of partial context labels w_t . w_t can be extracted from label annotations. For instance, in a single-label human activity classification dataset, w_t may contain a single phrase (e.g., {“walking”}), whereas in a multi-label dataset, it may include multiple phrases (e.g., {“at school”, “working on computers”}). We employ separate encoders for sensor and text inputs. Formally, let θ denote the complete sensor encoder model. The encoded sensor embedding is given by $z_t^s = \theta(x_t)$. The text encoder, denoted by ϕ , maps arbitrary phrases to a text embedding $z_t^w = \phi(w_t)$. Key notations used throughout our work are summarized in Table 4. We use distinct sensor encoder for each sensor modality (e.g., IMUs, audio and phone status) to accommodate the varying complexity of different sensor data types. For instance, a Transformer-based encoder is used for high-frequency time series data while a simple linear layer is designed for encoding phone status. The outputs from these modality-specific encoders are concatenated and passed through a fusion layer to generate the final

Table 4. List of important notations.

Symbol	Meaning	Symbol	Meaning
q^i, a^i	Question & answer pair	c^i	Extracted context from question decomposition
$\mathbf{x}_t$	Multimodal sensor data sampled at t	w_t	Partial context labels at t
d	Number of different sensors	T	Total duration of sensor data
θ	Sensor encoder	ϕ	Text encoder
$\mathbf{z}_t^s$	Sensor embedding of $\mathbf{x}_t$	$\mathbf{z}_t^w$	Text embedding of w_t
$A(w)$	Set of negative contexts of w	τ	Temperature scalar
f	Trained similarity function between embeddings	h	Threshold in query search

sensor embedding. Our framework allows for missing modalities by padding with mean values and is flexible for future expansion to additional sensor modalities.

We introduce a new pretraining loss to align sensor embeddings $\mathbf{z}_t^s$ and partial text embeddings $\mathbf{z}_t^w$. Different from CLIP [45], our loss function enables alignment between sensor data and all partial phrases:

$$\mathcal{L} = \sum_t \frac{-1}{|w_t|} \sum_{w \in w_t} \log \frac{\exp(\mathbf{z}_t^s \cdot \mathbf{z}_t^w / \tau)}{\sum_{a \in A(w)} \exp(\mathbf{z}_t^s \cdot \mathbf{z}_t^a / \tau)}, \quad (2)$$

$|w_t|$ is the cardinality of set w_t , τ is a scalar temperature parameter. The term $A(w) \equiv A \setminus \{w\}$ represents the collection of negative contexts, defined as all possible phrases except the positive phrase w . The intuition behind our loss function can be elaborated with a concrete example, as shown in Fig. 5. Consider a sensor embedding $\mathbf{x}_t$ and a set of partial text labels $w_t = \{\text{sitting, working on computers, at school}\}$. Our loss function encourages high similarity between the sensor embedding $\mathbf{z}_t^s$ and all positive text embeddings corresponding to w_t , namely "sitting", "working on computers", "at school", while distinguishing them from other negative contexts such as "walking". We draw inspiration from the supervised contrast learning loss [29]. However, our loss function differs in that it explicitly models the similarity between sensor embeddings and multiple text phrases rather than between samples of the same modality. After pretraining, we store all sensor embeddings in a database for online queries.

4.3.2 Online Sensor Data Query. SensorChat further introduces a query mechanism to extract relevant sensor embeddings and convert them into textual summarizations based on the pretrained sensor and text encoders. The active query process of SensorChat differs significantly from the passive journaling or summarization modules [42, 57], as SensorChat dynamically selects the query functions and query ranges based on the questions. The performance of sensor data query directly impacts the accuracy of SensorChat's answers. As shown in the middle part of Fig. 3, after SensorChat encodes the context c_i using the text encoder (④), SensorChat performs a similarity search among the sensor embedding given the text embedding (⑤), and summarizes the retrieved sensor embeddings into textual information using custom query functions (⑥). We next detail the similarity-based embedding search and query function design.

Similarity-based Embedding Search Our goal is to accurately identify all sensor samples relevant to the query context. For example, if the query context c_i is "exercise", SensorChat aims to retrieve all sensor samples associated with activities similar to exercise. If multiple text are generated during question decomposition, SensorChat performs an embedding search for each query text individually. The pretraining in Sec. 4.3.1 ensures that the sensor embedding space is well aligned with the text embedding space even using partial context. Therefore, the similarity comparison given any pair of sensor and text embeddings can be achieved by training a similarity function f in the embedding space:

$$f(\mathbf{z}_t^s, \mathbf{z}_t^w) = f(\theta(\mathbf{x}_t), \phi(w)) \in [0, 1] \quad (3)$$

The output of f is a scalar value between 0 and 1, representing the similarity between the sensor and text samples. With f , the similarity search in the embedding space is significantly more efficient than searching directly through raw, high-frequency time-series data. The efficiency can be further improved by narrowing the search scope based on the date and time-of-day arguments identified during question decomposition.

Query Function Design SensorChat uses a set of query functions to filter the relevant sensor samples and generate textual information for the answer assembly stage. The specific query function to be used is determined during question decomposition. For example, a question of "how long" should be directed to the CalculateDuration function, while a question of "what did I do" should be handled by the DetectActivity function. Each query function uses a unique template to return text information. The numerical value in the returned text is determined by the embedding search results. For instance, when querying CalculateDuration with the activity "cooking", date "Sunday", and time "morning", the textual output can be: "You spent γ minutes cooking on Sunday morning", where γ is calculated as follows:

$$\gamma = \sum_{t \in T_{\text{SundayMorning}}} \left[f(\theta(\mathbf{x}_t), \phi(\text{"cooking"})) > h \right]. \quad (4)$$

Here $T_{\text{SundayMorning}}$ represents all timestamps within Sunday morning. The notation $[Cond]$ gives 1, when the inner condition $Cond$ is met; otherwise 0. h is a predetermined threshold. In SensorChat, we carefully design a set of query functions to account for diverse scenarios in real life, including time queries, activity queries, counting, etc. The details of those functions are explained in Appendix B.

4.4 Answer Assembly

As shown in the right box of Fig. 3, the final stage of answer assembly integrates question and sensor information to generate the final answer. State-of-the-art methods [39, 55, 63] rely on training a neural network to fuse natural language and sensor data, similar to a "black-box". These methods often lead to ineffective fusion and inaccurate answers when dealing with long-duration, high-frequency sensor data (see Sec. 3.2). In contrast, SensorChat directly fuses the filtered sensor context from the query stage with the original question. Our intuition is that, in contrast to processing and fusing with other modalities, LLMs are the most professional in dealing with text. SensorChat is capable of answering both qualitative and quantitative questions by combining the original question and the extracted fine-grained activity information from long-duration, high-frequency sensors. At this answer assembly stage, we finetune a LLM such as LLaMA [63] to adapt the model to the desired answer style. Fine-tuning is chosen over few-shot learning as it delivers better performance with the presence of high-quality datasets like SensorQA [47], which is verified in Appendix F. We use Low Rank Adaptation (LoRA) [24] due to its parameter efficiency and comparable performance to full fine-tuning. The prompt is as follows: "Answer the question based on the context below. Context: {Context} Question: {Question} Response:"

5 SYSTEM IMPLEMENTATION

We implement SensorChat on real-world systems². We envision SensorChat as a personal assistant that provides accurate and timely answers to questions from the users, as outlined in our problem statement (Sec. 3.1). Fig. 6 visualizes the general system pipeline of SensorChat. We employ smartphones and smartwatches to collect multimodal sensor data from users in daily lives (① in Fig. 6a). Implemented based on the ExtraSensory App [52], the mobile devices automatically gathers data for 20 seconds every minute, including 40Hz IMU signals, 13 MFCC audio features from a 22kHz microphone, and other phone state information including compass, GPS location, Wi-Fi status, light intensity, battery level, etc. The details can be found in the ExtraSensory App manual [52]. These sensor data are then transmitted to a system running SensorChat (② in Fig. 6a). We offer two variants of

²Our code is available at <https://github.com/Orienfish/SensorChat>.

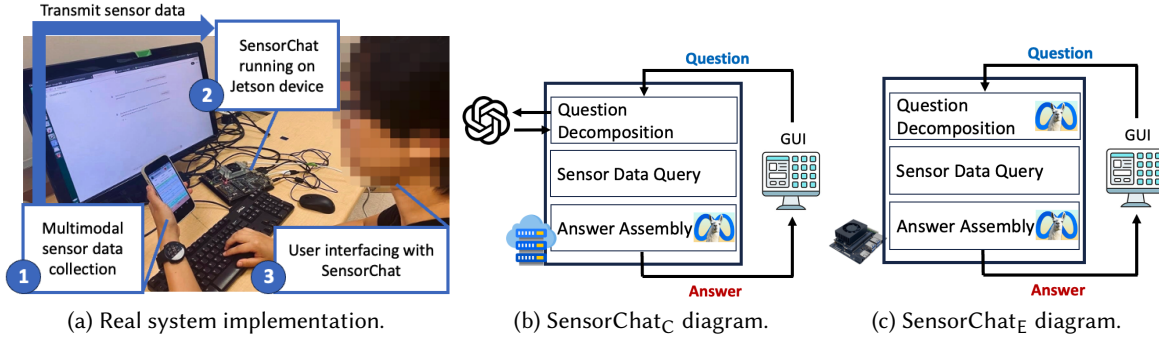


Fig. 6. System implementation of SensorChat.

SensorChat, designed for a cloud server and an edge environment respectively. Their detailed implementations and trade-offs are discussed below. Finally, users can interact with SensorChat directly through a chatting interface using natural language, shown as ③ in Fig. 6a.

SensorChat_C and SensorChat_E We offer two system variants of SensorChat, as shown in Fig. 6b and 6c.

- **SensorChat_C**, designed for a cloud environment, uses GPT-3.5-Turbo [61] for question decomposition and a full-size finetuned LLaMA2-7B model [50] for answer assembly. We deploy and test SensorChat on a cloud server equipped with an NVIDIA A100 GPU [3].
- **SensorChat_E**, designed for an edge environment, uses quantized LLaMA model [50] for both question decomposition and answer assembly. The question decomposition model is quantized from the official LLaMA3-8B, while the answer assembly model is quantized from our fine-tuned version of LLaMA2-7B. We use Activation-aware Weight Quantization (AWQ), a state-of-the-art quantization method for LLMs, known for its hardware efficiency. We deploy and test SensorChat_E on a NVIDIA Jetson Orin NX module [2] with 16GB RAM.

SensorChat_C and SensorChat_E accommodate two typical use scenarios. SensorChat_C is expected to deliver superior QA performance with the full-precision LLMs in the cloud, at the cost of intensive resource consumption. Additionally, SensorChat_C requires the users to transmit the full sensor history to the cloud server. On the other hand, SensorChat_E runs entirely on a local edge platform belonging to the user, eliminating the need to transmit user data to the cloud and thus preserving user privacy. However, its QA and latency performance degrade compared to SensorChat_C. Further implementation details can be found in Appendix C.

6 EVALUATION ON STATE-OF-THE-ART DATASET

In this section, we thoroughly evaluate SensorChat on the state-of-the-art dataset focusing on quantitative questions. Due to space limitations, we defer the ablation study, sensitivity analysis, and generalization study to Appendix F–I. We further validate SensorChat in a real world study with both quantitative and qualitative questions in Sec. 7.

6.1 Dataset and Metrics

In this evaluation section, we focus mainly on the SensorQA dataset [47] and quantitative questions. To the best of our knowledge, SensorQA is the first and only available benchmarking dataset for QA interactions that use long-term timeseries sensor data and reflect practical user interests. While we focus on SensorQA [47] in this section, we emphasize that SensorChat is broadly applicable and can be extended to other real-world sensing applications. To ensure the best alignment between the QA pairs and sensor information, we conduct offline

encoders pretraining on the ExtraSensory multimodal sensor dataset [51], which serves as the sensor data source for SensorQA. During pretraining, all sensor samples are aligned by a time window of 20 seconds.

Dataset Variants and Metrics We evaluate three versions of SensorQA [47] using various metrics to assess both the quality and accuracy of the generated answers.

- **Full answers** refer to the original complete responses in SensorQA. We evaluate the model's performance on the full answers using Rouge-1, Rouge-2, and Rouge-L scores [18]. Rouge scores measure the overlap of n-grams between the machine-generated content and the ground-truth answers, expressed as F-1 scores. Higher Rouge scores indicate greater similarity between the generated and true answers.
- **Short answers** are the 1-2 key words extracted from the full answers by GPT-3.5-Turbo [61], offered along with the original SensorQA dataset [47]. We use the exact match accuracy on the short answers to evaluate the precision of generated answers, as detailed in Sec. 3.2.
- **Multiple choices** are generated by prompting GPT-3.5-Turbo [61] to create three additional choices similar to the correct short answer. An example QA can be "*Which day did I spend the most time with coworkers?* A. Friday, B. Monday, C. Thursday, D. Wednesday", with the correct answer being "D" or "D. Wednesday." The models are expected to accurately select the correct answer from the four candidates. We evaluate the performance based on exact answer selection accuracy.

We create the multiple choices in addition to the full answers and short answers provided in the original SensorQA dataset [47] to further assess the models' ability in distinguishing similar facts based on sensor data.

6.2 State-of-the-Art Baselines

We compare SensorChat against the state-of-the-art baselines that leverage different modalities combinations, including **text**, **vision+text**, and **sensor+text** data. For the text-only models, we employ the popular **GPT-4** [5], **T5** [46] and **LLaMA** [50] to obtain a baseline performance without considering the sensor data. For the vision+text models, we adopt **GPT-4-Turbo** [5], **GPT-4o** [5], **LLaMA-Adapter** [63] and **LLaVA-1.5** [33]. We feed the activity graphs in SensorQA [47] (similar to Fig. 1c) into these models along with the questions as input. We also compare against the latest sensor+text models including **IMU2CLIP+GPT-4o** [39], **LLMSense** [42], **DeepSQA** [55] and **OneLLM** [21]. To ensure a fair comparison, we adapt these baselines to process the same long-duration, high-frequency data by modifying the data format to match their original input modality. Further adaptation details are provided in Appendix D.

For the closed-source generative baselines based on GPT, we use few-shot learning (FSL). Specifically, we incorporate a set of QA examples from SensorQA [47] into the prompt for each question input. We adopt 10 samples per question based on a grid search of {2, 5, 10, 15}. For T5 and DeepSQA, we train the models directly on the SensorQA dataset [47]. For open source LLM baselines such as LLaMA and LLaMA-Adapter, we apply LoRA fine-tuning (FT) [24] using the samples from SensorQA [47]. We randomly select 80% of the QA pairs in SensorQA [47] as training samples and reserve the remaining 20% for testing. We explore alternative splitting schemes in Appendix I to demonstrate SensorChat's generalizability to unseen users. All baselines based on LLaMA (that is, LLaMA, LLaMA-Adapter, and OneLLM) use LLaMA2-7B [50]. All baselines adopt the same hyperparameters as those specified in their official codebases.

6.3 QA Performance

We start with presenting an example of SensorChat compared to top-performing baselines in Fig. 7. Additional QA examples are presented in Appendix E. Answering the question in Fig. 7 involves two steps: counting meal frequency per day and identifying the day with the fewest meals. All three baselines fail in this challenging task. DeepSQA [55] and LLaMA-Adapter [63] struggle with long-duration, high-frequency data, as they were originally designed for short sensor inputs under one minute. In our setting, these models fail to learn effective fusion

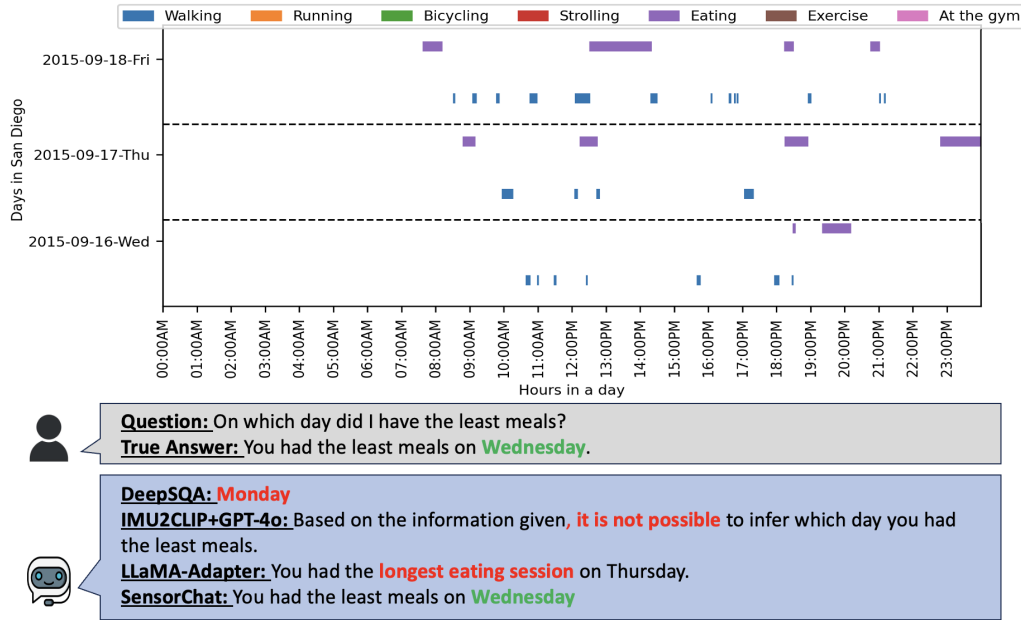


Fig. 7. Qualitative results of SensorChat in comparison to state-of-the-art methods on SensorQA [47]. Key phrases in the answers are highlighted in the green if they match the true answer, and in the red if they do not. SensorChat is the only method among the presented baselines that correctly interprets the question and generates an accurate answer.

between long-term sensor data and textual questions, leading to wrong answers. IMU2CLIP+GPT-4o [39] works by first mapping raw sensor signals into a long narrative log and then passing the log to GPT-4o for reasoning. Although IMU2CLIP+GPT-4o is able to accurately associate sensor signals with text, it is constrained by GPT-4o's token limit, causing key portions of the log to be truncated and leading to incomplete or uninformative answers. In contrast, SensorChat succeeds by decomposing the query in the first stage, extracting daily meal frequencies in the second stage, and leaving the comparison to the answer stage. The collaboration across the three stages allows SensorChat to effectively handle quantitative queries that require precise numerical reasoning across multi-day span, which highlights SensorChat's advancements over existing works.

Table 5 presents the comprehensive results of all methods on the three variants of SensorQA [47]: full answers, short answers, and multiple choices. The full answers dataset evaluates overall language quality, while the short answers and multiple choices focus on the models' ability to learn underlying facts and numerical results rather than the language itself. Our SensorChat outperforms the best state-of-the-art methods with the **highest Rouge scores on full answers, 93% higher accuracy on short answers, and 25% higher accuracy on multiple choices**. SensorChat_C performs slightly better than SensorChat_E mainly due to the stronger model capability of GPTs compared to quantized LLaMA in question decomposition. However, SensorChat_E, as a pure edge solution, preserves better user privacy as discussed in Sec. 5. The gains of SensorChat over the baselines are more significant for short answers/multiple choices than for full answers, highlighting SensorChat's particular advantage in factual and numerical reasoning in comparison to existing works. It is important to note that the exact match accuracy for short and multiple choice answers is a strict metric, as it requires the model to generate the *exact* answer as the correct one. For instance, "4 hours" and "3 hours 50 min" would be considered different. Answering multiple-choice questions can also be particularly challenging, as candidate answers differ only slightly, such as "A. 10 min" vs. "B. 20 min". However, in practical applications, a QA system does not necessarily need to achieve

Modalities	Method	FSL/FT ¹	Full Answers			Short Answers	Multiple Choices
			Rouge-1 (↑)	Rouge-2 (↑)	Rouge-L (↑)	Accuracy (↑)	Accuracy (↑)
Text	GPT-4 [5]	FSL	0.66	0.51	0.64	0.16	0.34
Text	T5-Base [46]	FT	0.71	0.55	0.69	0.25	0.52
Text	LLaMA2-7B [50]	FT	0.72	0.62	0.72	0.26	0.56
Vision+Text	GPT-4-Turbo [5]	FSL	0.38	0.28	0.36	0.14	0.24
Vision+Text	GPT-4o [5]	FSL	0.39	0.28	0.37	0.20	0.07
Vision+Text	LLaMA-Adapter [63]	FT	0.73	0.57	0.71	0.28	0.54
Vision+Text	LLaVA-1.5 [33]	FT	0.62	0.46	0.60	0.21	0.47
Sensor+Text	IMU2CLIP+GPT4o [39]	FSL	0.58	0.44	0.55	0.28	0.49
Sensor+Text	LLMSense [42]	FSL	0.37	0.26	0.35	0.23	0.49
Sensor+Text	DeepSQA [55]	FT	0.34	0.05	0.34	0.27	-
Sensor+Text	OneLLM [21]	FT	0.12	0.04	0.12	0.05	0.30
Sensor+Text	SensorChat_C	FT	0.77	0.62	0.75	0.54	0.70
Sensor+Text	SensorChat_E	FT	0.76	0.60	0.74	0.49	0.67

¹FSL: Few-Shot Learning. FT: Finetuning.

Table 5. Comprehensive results of SensorChat compared against state-of-the-art methods on SensorQA [47]. Bold and underlined values show the best results (all achieved by SensorChat) and the best among baselines. DeepSQA [55] was not evaluated on the multiple-choice version due to its inability to handle dynamic answer choices.

perfect exact match accuracy to be useful. We conduct a user study in Sec. 7 to evaluate SensorChat in real-world scenarios. Even under the strict exact match evaluation, SensorChat achieves 54% accuracy on short answers and 70% on multiple choices. **These accuracy improvements demonstrate SensorChat’s effectiveness in learning the underlying life facts from the long-duration, multimodal time series sensor data.**

In contrast, all baseline methods struggle with quantitative accuracy, achieving at most only 28% on short-answer questions. Surprisingly, text-only baselines like LLaMA2-7B [50] and T5 [46] achieve some of the highest accuracy among the baselines, with 27% accuracy on short answers. The limited improvements from adding vision or sensor modalities suggest that existing baselines fail to effectively fuse multimodal information in our setting. There are several reasons for this underperformance. GPT-based models (e.g., GPT-4-Turbo and GPT-4o [5]) are not trained for interpreting sensor data or reasoning about daily activities. Their lower Rouge scores are often due to overly long answers that deviate from the ground truth. Vision+text models like LLaMA-Adapter [63] and LLaVA-1.5 [33] were originally developed for video captioning and lack the precision needed for quantitative reasoning in daily life activities. For sensor+text baselines, IMU2CLIP+GPT-4o [39] and LLMSense [42] rely on feeding long narrative logs to LLMs to provide the sensor information necessary for accurately answering the questions, making them vulnerable to LLMs’ well-known struggles with long-context reasoning [20, 25]. Others like DeepSQA [55] and OneLLM [21] were built for short-duration data (i.e., less than one minute) and cannot scale effectively to long-duration, high-frequency inputs of multiple days with their current model and training pipeline. In contrast, SensorChat overcomes these limitations by introducing a dedicated query stage that retrieves relevant sensor embeddings from a pre-encoded database. This design enables accurate answers to quantitative questions in SensorQA [47].

6.4 End-to-End Answer Generation Latency

To evaluate efficiency, we measure the end-to-end answer generation latency of SensorChat_C and SensorChat_E on their respective platforms - cloud server and NVIDIA Jetson Orin. For a fair comparison, we quantize all LLM-based baselines to 4-bit weights using AWQ [32] for the Jetson Orin experiments, matching the setup of SensorChat_E. **SensorChat takes an average generation latency of 2.3s on the cloud and 10.0s on the**

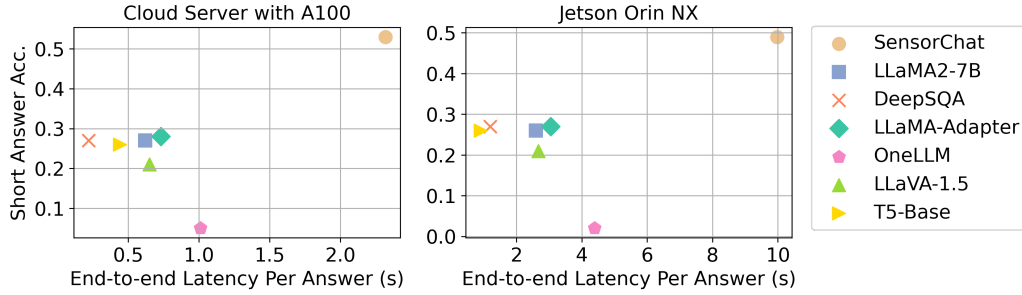


Fig. 8. The answer accuracy and latency trade-offs of all methods measured on the cloud and edge platforms. We evaluate SensorChat_C on A100 [3] and SensorChat_E on Jetson Orin NX [2]. All LLM-based models are quantized to 4-bit weights with AWQ [32] for Jetson Orin deployments.

Jetson Orin. Specifically, SensorChat_C takes an average of 1.2s for question decomposition, 0.5s for sensor data query, and 0.6s for answer assembly. SensorChat_E takes an average of 2.5s, 4.9s, and 2.5s for each stage, respectively.

The accuracy-latency trade-off of SensorChat and locally running baselines are shown in Fig. 8. Despite higher latency due to its dual-LLM design, SensorChat outperforms other baselines in accuracy. SensorChat still achieves real-time responses on the cloud and can be deployed on resource-constrained devices with reasonable generation latency. Notably, quantization causes negligible accuracy loss in SensorChat_E compared to the full-precision SensorChat_C. This is due to SensorChat’s design, which converts sensor data into text, making the final answer assembly a purely language-based task where techniques like AWQ [32] minimize accuracy degradation in language tasks after quantization. We plan to further optimize the latency of SensorChat on edge devices via algorithm-system co-design in future work.

7 REAL USER STUDY

Since the evaluations in Sec. 6 focus on quantitative questions, we further conduct a user study to assess whether SensorChat can assist users in real-world scenarios. This real user study evaluates SensorChat’s generalizability to personalized sensor setups and to arbitrary user-generated questions by having the participants collect their own sensor data and create questions of personal interest. Our study is approved by the Institutional Review Board Committee.

User Study Setup We recruited eight volunteers (five males and three females) who were instructed to follow their normal daily routines while carrying a smartphone with the ExtraSensoryApp [52]. The smartphone models include Huawei Mate 10 Pro, LG G7 ThinQ and Google Pixel 2. The app automatically collected multimodal sensor data and transmitted it to SensorChat whenever a network connection was available. Reporting activity labels was optional for the participants. The data collection phase lasted one to three days, with valid samples ranging from 52 to 1366 minutes. Sensor data availability varied due to factors such as phone model and usage patterns. After the data collection phase, volunteers were invited to interact with SensorChat_C in-person through a chat-based graphical user interface (GUI). Details of the GUI are provided in Appendix J. They were encouraged to ask *any* questions about their lives during the data collection phase and observe SensorChat_C’s response generation in real time. In the final step, we gathered feedback from the participants including ratings on answer content, latency, and practical value of SensorChat on a scale from 1 to 5. We specifically select SensorChat_C for the user study to evaluate SensorChat’s full functionality in real-world conditions. We leave user evaluation and optimization of SensorChat_E in a purely edge scenario for future work.

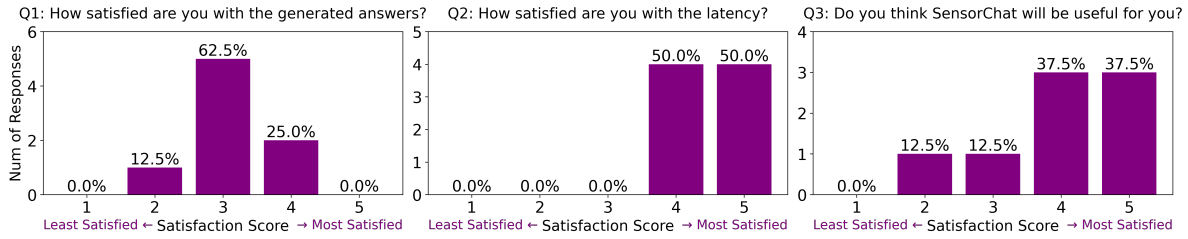


Fig. 9. Feedback ratings from eight participants on answer satisfaction (left), latency satisfaction (middle), and the usefulness of SensorChat (right). Participants recognized SensorChat's answer quality (on both quantitative and qualitative questions) along with its low latency and practical value.

User Feedback Results Fig. 9 shows the feedback ratings from eight participants. **SensorChat received an average score of 3.12 for answer content, 4.50 for latency, and 4.00 for practical utility**, highlighting its potential for real-world use. We observed that users asked a wide range of questions, including both quantitative and qualitative ones. While some of the questions raised by participants overlap with the quantitative questions in SensorQA [47], many are qualitative in nature and are not covered by the dataset. The 3.12 average score suggests that SensorChat can handle both types of questions, which is a difficult task. More details of the questions asked by the users are presented in Appendix K. Participants appreciated the natural tone of the responses, such as “*The answers were formatted in an easy to understand way.*” Some concerns were noted about accuracy, including “*some numbers were a little off*” and “*it mentioned activities I never did.*” We believe these issues stem from the limited ability of the sensor encoder to generalize from the dataset to real-world users. In several cases, the sensor query stage produced noisy outputs and occasionally detected activities that did not occur. Fortunately, generalizing models across different settings is a well-studied problem [58]. Applying these techniques to SensorChat may improve its real-world performance, which we leave for future work.

In terms of latency, all participants give positive feedbacks regarding the end-to-end answer generation latency, with comments such as “*I do not feel as if I had to wait a long time for the answers*” and “*I think it is faster than I thought previously*”. Most participants are positive in terms of the practical utility of SensorChat in their daily lives, e.g., “*I believe it can help with my wellness management significantly.*” The less positive comments mentioned the challenge of adapting SensorChat to individual users for generating more personalized and useful responses, an issue that we leave for future exploration.

Generalization to Qualitative and Open-Ended Questions Participants have asked qualitative and open-ended questions to SensorChat that do not exist in SensorQA, providing a more comprehensive assessment of SensorChat. For example, one participant asked, “*How would you rate my lifestyle and what improvements do you suggest?*” SensorChat responded with, “*I would rate this lifestyle as 2 out of 5 stars and would suggest you improve on your exercise and talking,*” based on the duration of each activity where exercise and talking were lacking. One advantage of SensorChat is that it can handle qualitative questions with the same three-stage pipeline. SensorChat effectively handles qualitative questions by comprehensively querying a list of contexts that are relevant to the qualitative question, such as “sitting”, “exercising”, and “talking” in the above example. SensorChat then passes the full set of context information to the answer assembly stage, where the LLM uses both this context and its world knowledge to reason about the user’s lifestyle. In our study, participants appreciated the ability of SensorChat to effectively handle both quantitative and qualitative questions, e.g., “*I think it could be very useful to be able to answer these qualitative and quantitative questions about my lifestyle,*” highlighting its broad applicability to diverse real-life scenarios. We plan to conduct a larger-scale user study with a broader participant pool in the future to further investigate the generalizability of SensorChat across diverse age and gender groups.

8 DISCUSSION AND FUTURE WORK

Generalizability of SensorChat In this work, we design SensorChat address two main categories of questions: quantitative and qualitative. SensorChat provides accurate answers to quantitative questions by a successful fusion between LLMs and a sensor data query stage. The LLM interprets the question and generates natural language answers, while the query stage extracts relevant context from long-duration, high-frequency sensor data. Using the same pipeline, SensorChat also handles qualitative questions by passing richer context summaries to the LLM for high-level reasoning. However, SensorChat can still produce imprecise answers for time-related queries. Future work may explore more advanced LLMs, such as OpenAI’s o3-mini [4] or DeepSeek [1], which perform better in numerical reasoning. SensorChat can also incorporate more sensors via adding additional modality encoders. Additionally, SensorChat can serve a larger user base by collecting more data and equipping with better generalization designs such as [58]. SensorChat is a general and flexible framework designed for fusing long-duration multimodal sensors knowledge with language and answering versatile questions.

Evaluation Metrics SensorChat uses traditional metrics in natural language processing and the precision of the answers in evaluation. While these metrics excel in objectiveness, they do not capture subjective user satisfaction, which is crucial for assessing a system’s readiness for a broad market. We leave a comprehensive evaluation of SensorChat with subjective metrics as our future work.

Efficiency of SensorChat In this work, we demonstrate that SensorChat can be deployed on Jetson-level edge devices using common quantization techniques. However, further optimization is needed for real-time LLM service on edge devices. The long-term goal for systems like SensorChat is to run on mobile devices. Recent studies [35, 66] have explored enabling real-time LLM serving on mobile devices, and these techniques could be integrated into SensorChat in future work. Additionally, we recognize the potential synergy between SensorChat and vector databases [65] or retrieval-augmented generation [64]. The techniques in these domains can be integrated into SensorChat to improve query performance and efficiency.

9 CONCLUSION

Natural interactions between users and multimodal sensors can unlock the full potential of sensor data in real-world applications. However, existing systems struggle to process long-duration, high-frequency sensor data, often resulting in unsatisfactory answers to long-term questions. In this paper, we present SensorChat, the first end-to-end system designed to handle long-duration, high-frequency time series sensor data for both qualitative and quantitative question answering. SensorChat introduces a novel three-stage pipeline including question decomposition, sensor data query and answer assembly. All stages and interfaces are carefully designed to effectively fuse the natural language and the sensor modality. Evaluation results and user studies demonstrate SensorChat’s effectiveness in answering a wide range of qualitative and quantitative questions from state-of-the-art benchmarks and a real-world user study. The cloud version of SensorChat system supports real-time interactions, while the edge version is optimized to run on Jetson-level devices.

Acknowledgments

This work was supported in part by National Science Foundation under Grants #2003279, #2112167, #1911095, #2112665, #2120019, #2211386 and in part by PRISM and CoCoSys, centers in JUMP 2.0, a SRC program sponsored by DARPA.

References

- [1] 2025. DeepSeek. <https://www.deepseek.com/>. [Online].
- [2] 2025. Jetson Orin NX Module. https://developer.nvidia.com/embedded/jetson-modules#jetson_orin_nx. [Online].
- [3] 2025. NVIDIA A100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/a100/>. [Online].
- [4] 2025. OpenAI o3-mini. <https://openai.com/index/openai-o3-mini/>. [Online].

- [5] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [6] Deepa Tavargeri Adiga, Maitry Bhavsar, Unnati Palan, and Sachin Patel. 2020. Daily Journals: Extracting Insights for Well-being. In *Proceedings of the 14th EAI International Conference on Pervasive Computing Technologies for Healthcare*. 305–315.
- [7] Kiyoharu Aizawa, Datchakorn Tancharoen, Shinya Kawasaki, and Toshihiko Yamasaki. 2004. Efficient retrieval of life log based on context and content. In *Proceedings of the 1st ACM workshop on Continuous archival and retrieval of personal experiences*. 22–31.
- [8] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems* 35 (2022), 23716–23736.
- [9] Riku Arakawa, Jill Fain Lehman, and Mayank Goel. 2024. PrISM-Q&A: Step-Aware Voice Assistant on a Smartwatch Enabled by Multimodal Procedure Tracking and Large Language Models. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8, 4 (2024), 1–26.
- [10] Qiming Cao, Hongfei Xue, Tianci Liu, Xingchen Wang, Haoyu Wang, Xincheng Zhang, and Lu Su. 2024. mmCLIP: Boosting mmWave-based Zero-shot HAR via Signal-Text Alignment. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems*. 184–197.
- [11] Wenqiang Chen, Jiaxuan Cheng, Leyao Wang, Wei Zhao, and Wojciech Matusik. 2024. Sensor2Text: Enabling Natural Language Interactions for Daily Activity Tracking Using Wearable Sensors. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8, 4 (2024), 1–26.
- [12] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. <https://lmsys.org/blog/2023-03-30-vicuna/>
- [13] Wonyoung Choi, Jisu Kim, SangEun Lee, and Eunil Park. 2021. Smart home and internet of things: A bibliometric study. *Journal of Cleaner Production* 301 (2021), 126908.
- [14] Ranak Roy Chowdhury, Ritvik Kapila, Ameiya Panse, Xiyuan Zhang, Diyan Teng, Rashmi Kulkarni, Dezhi Hong, Rajesh K Gupta, and Jingbo Shang. 2025. ZeroHAR: Sensor Context Augments Zero-Shot Wearable Action Recognition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 16046–16054.
- [15] Zheng Chu, Jingchang Chen, Qianglong Chen, Weijiang Yu, Tao He, Haotian Wang, Weihua Peng, Ming Liu, Bing Qin, and Ting Liu. 2024. Navigate through Enigmatic Labyrinth A Survey of Chain of Thought Reasoning: Advances, Frontiers and Future. In *The 62nd Annual Meeting of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, August 11–16, 2024*. Association for Computational Linguistics. <https://arxiv.org/abs/2309.15402>
- [16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [17] Zachary Enghardt, Chengqian Ma, Margaret E Morris, Chun-Cheng Chang, Xuhai" Orson" Xu, Lianhui Qin, Daniel McDuff, Xin Liu, Shwetak Patel, and Vikram Iyer. 2024. From Classification to Clinical Insights: Towards Analyzing and Reasoning About Mobile and Behavioral Health Data With Large Language Models. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8, 2 (2024), 1–25.
- [18] Matan Eyal, Tal Baumel, and Michael Elhadad. [n.d.]. Question Answering as an Automatic Evaluation Metric for News Article Summarization. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 3938–3948.
- [19] Yonggan Fu, Yongan Zhang, Zhongzhi Yu, Sixu Li, Zhifan Ye, Chaojian Li, Cheng Wan, and Yingyan Celine Lin. 2023. Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [20] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).
- [21] Jiaming Han, Kaixiong Gong, Yiyuan Zhang, Jiaqi Wang, Kaipeng Zhang, Dahua Lin, Yu Qiao, Peng Gao, and Xiangyu Yue. 2024. Onellm: One framework to align all modalities with language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.
- [22] Gerhard Henrich and Peter Herschbach. 2000. Questions on Life Satisfaction (FLZM): a short questionnaire for assessing subjective quality of life. *European Journal of Psychological Assessment* 16, 3 (2000), 150.
- [23] Aritra Hota, Soumyajit Chatterjee, and Sandip Chakraborty. 2024. Evaluating large language models as virtual annotators for time-series physical sensing data. *arXiv preprint arXiv:2403.01133* (2024).
- [24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).
- [25] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232* (2023).

- [26] Keum-Sung Hwang and Sung-Bae Cho. 2014. A Lifelog Browser for Visualization and Search of Mobile Everyday-Life. *Mobile Information Systems* 10, 3 (2014), 243–258.
- [27] Sijie Ji, Xinzhe Zheng, Jiawei Sun, Renqi Chen, Wei Gao, and Mani Srivastava. 2024. MindGuard: Towards Accessible and Sitgma-free Mental Health First Aid via Edge LLM. *arXiv preprint arXiv:2409.10064* (2024).
- [28] Sijie Ji, Xinzhe Zheng, and Chenshu Wu. 2024. Hargpt: Are llms zero-shot human activity recognizers?. In *2024 IEEE International Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things (FMSys)*. IEEE, 38–43.
- [29] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. *Advances in neural information processing systems* 33 (2020), 18661–18673.
- [30] Yubin Kim, Xuhai Xu, Daniel McDuff, Cynthia Breazeal, and Hae Won Park. 2024. Health-llm: Large language models for health prediction via wearable sensor data. *arXiv preprint arXiv:2401.06866* (2024).
- [31] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. 2024. Long-context llms struggle with long in-context learning. *arXiv preprint arXiv:2404.02060* (2024).
- [32] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *MLSys*.
- [33] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. 2024. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 26296–26306.
- [34] Xin Liu, Daniel McDuff, Geza Kovacs, Isaac Galatzer-Levy, Jacob Sunshine, Jiening Zhan, Ming-Zher Poh, Shun Liao, Paolo Di Achille, and Shwetak Patel. 2023. Large language models are few-shot health learners. *arXiv preprint arXiv:2305.15525* (2023).
- [35] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, et al. 2024. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. *arXiv preprint arXiv:2402.14905* (2024).
- [36] Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. 2022. Learn to explain: Multimodal reasoning via thought chains for science question answering. *Advances in Neural Information Processing Systems* 35 (2022), 2507–2521.
- [37] Shentong Mo, Russ Salakhutdinov, Louis-Philippe Morency, and Paul Pu Liang. 2024. Iot-llm: Large multisensory language models for the internet of things. *arXiv preprint arXiv:2407.09801* (2024).
- [38] Seungwhan Moon, Andrea Madotto, Zhaojiang Lin, Tushar Nagarajan, Matt Smith, Shashank Jain, Chun-Fu Yeh, Prakash Murugesan, Peyman Heidari, Yue Liu, et al. 2023. Anymal: An efficient and scalable any-modality augmented language model. *arXiv preprint arXiv:2309.16058* (2023).
- [39] Seungwhan Moon, Andrea Madotto, Zhaojiang Lin, Aparajita Saraf, Amy Bearman, and Babak Damavandi. 2023. IMU2CLIP: Language-grounded Motion Sensor Translation with Multimodal Contrastive Learning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 13246–13253.
- [40] Jingping Nie, Minghui Zhao, Stephen Xia, Xinghua Sun, Hanya Shao, Yang Fan, Matthias Preindl, and Xiaofan Jiang. 2022. Ai therapist for daily functioning assessment and intervention using smart home devices. In *Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems*. 764–765.
- [41] Xiaomin Ouyang, Xian Shuai, Jiayu Zhou, Ivy Wang Shi, Zhiyuan Xie, Guoliang Xing, and Jianwei Huang. 2022. Cosmo: contrastive fusion learning with small data for multimodal human activity recognition. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*. 324–337.
- [42] Xiaomin Ouyang and Mani Srivastava. 2024. LLMsense: Harnessing LLMs for high-level reasoning over spatiotemporal sensor traces. In *2024 IEEE 3rd Workshop on Machine Learning on Edge in Sensor Systems (SenSys-ML)*. IEEE, 9–14.
- [43] Xiaomin Ouyang, Zhiyuan Xie, Heming Fu, Sitong Cheng, Li Pan, Neiwen Ling, Guoliang Xing, Jiayu Zhou, and Jianwei Huang. 2023. Harmony: Heterogeneous Multi-Modal Federated Learning through Disentangled Model Training. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*. 530–543.
- [44] Adam Paszke et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [45] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [46] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67. <http://jmlr.org/papers/v21/20-074.html>
- [47] Benjamin Reichman, Xiaofan Yu, Lanxiang Hu, Jack Truxal, Atishay Jain, Rushil Chandrupatla, Tajana S Rosing, and Larry Heck. 2025. SensorQA: A Question Answering Benchmark for Daily-Life Monitoring. In *Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems*. 282–289.

- [48] Zhenwei Shao, Zhou Yu, Meng Wang, and Jun Yu. 2023. Prompting large language models with answer heuristics for knowledge-based visual question answering. In *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*. 14974–14983.
- [49] Kohei Takahashi, Shinsuke Matsumoto, Sachio Saiki, and Masahide Nakamura. 2013. Design and evaluation of lifelog mashup platform with NoSQL database. In *Proceedings of International Conference on Information Integration and Web-based Applications & Services*. 133–139.
- [50] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [51] Yonatan Vaizman, Katherine Ellis, and Gert Lanckriet. 2017. Recognizing detailed human context in the wild from smartphones and smartwatches. *IEEE pervasive computing* 16, 4 (2017), 62–74.
- [52] Yonatan Vaizman, Katherine Ellis, Gert Lanckriet, and Nadir Weibel. 2018. Extrasensory app: Data collection in-the-wild with rich user interface to self-report behavior. In *Proceedings of the 2018 CHI conference on human factors in computing systems*. 1–12.
- [53] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- [54] Yuxuan Weng, Guoquan Wu, Tianyue Zheng, Yanbing Yang, and Jun Luo. 2024. Large Model for Small Data: Foundation Model for Cross-Modal RF Human Activity Recognition. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems*. 436–449.
- [55] Tianwei Xing, Luis Garcia, Federico Cerutti, Lance Kaplan, Alun Preece, and Mani Srivastava. 2021. DeepSQA: Understanding Sensor Data via Question Answering. In *Proceedings of the International Conference on Internet-of-Things Design and Implementation*. 106–118.
- [56] Huatao Xu, Liying Han, Qirui Yang, Mo Li, and Mani Srivastava. 2024. Penetrative ai: Making llms comprehend the physical world. In *Proceedings of the 25th International Workshop on Mobile Computing Systems and Applications*. 1–7.
- [57] Huatao Xu, Panrong Tong, Mo Li, and Mani Srivastava. 2024. AutoLife: Automatic Life Journaling with Smartphones and LLMs. *arXiv preprint arXiv:2412.15714* (2024).
- [58] Huatao Xu, Pengfei Zhou, Rui Tan, and Mo Li. 2023. Practically Adopting Human Activity Recognition. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*. 1–15.
- [59] Lilin Xu, Chaojie Gu, Rui Tan, Shibo He, and Chen Jiming. 2023. MESEN: Exploit Multimodal Data to Design Unimodal Human Activity Recognition with Few Labels. In *Proceedings of the 21st ACM Conference on Embedded Networked Sensor Systems*.
- [60] Bufang Yang, Siyang Jiang, Lilin Xu, Kaiwei Liu, Hai Li, Guoliang Xing, Hongkai Chen, Xiaofan Jiang, and Zhenyu Yan. 2024. DrHouse: An LLM-empowered Diagnostic Reasoning System through Harnessing Outcomes from Sensor Data and Expert Knowledge. *arXiv preprint arXiv:2405.12541* (2024).
- [61] Junjie Ye, Xuanning Chen, Nuo Xu, Can Zu, Zekai Shao, Shichun Liu, Yuhuan Cui, Zeyang Zhou, Chao Gong, Yang Shen, et al. 2023. A comprehensive capability analysis of gpt-3 and gpt-3.5 series models. *arXiv preprint arXiv:2303.10420* (2023).
- [62] Hyungjun Yoon, Biniyam Aschalew Tolera, Taesik Gong, Kimin Lee, and Sung-Ju Lee. 2024. By my eyes: Grounding multimodal large language models with sensor data via visual prompting. *arXiv preprint arXiv:2407.10385* (2024).
- [63] Renrui Zhang, Jiaming Han, Chris Liu, Peng Gao, Aojun Zhou, Xiangfei Hu, Shilin Yan, Pan Lu, Hongsheng Li, and Yu Qiao. 2023. Llama-adapter: Efficient fine-tuning of language models with zero-init attention. *arXiv preprint arXiv:2303.16199* (2023).
- [64] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, and Bin Cui. 2024. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473* (2024).
- [65] Xuanhe Zhou, Xinyang Zhao, and Guoliang Li. 2024. LLM-Enhanced Data Management. *arXiv preprint arXiv:2402.02643* (2024).
- [66] Yan Zhuang, Zhenzhe Zheng, Fan Wu, and Guihai Chen. 2024. LiteMoE: Customizing On-device LLM Serving via Proxy Submodel Tuning. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems*. 521–534.

A Different Ways of Feeding Sensor Data to LLMs

Existing works have mainly used four ways to feed sensor data to LLMs: (1) converting raw sensor data into text and feeding to LLMs [17, 23, 28, 30, 34, 56, 60], (2) visualizing raw sensor signals in images and feeding to LLMs [62], (3) processing raw sensor signals with a pretrained sensor encoder and feeding the encoded sensor embeddings to LLMs [11, 21, 37, 38], and (4) processing raw sensor signals with a pretrained sensor encoder that produces classification results in text, and then feeding the narrative text to LLMs [9, 39, 42]. In the following lines, we explain why we choose (4) as the base framework for SensorChat, and why the first three approaches are not suitable for long-duration (more than one day), high-frequency (e.g., raw time series from IMU sensors) sensor data in our setting, as detailed in Sec. 1.

- (1) One common approach is to directly convert raw sensor measurements into text before passing them to LLMs, as seen in several recent works [17, 23, 28, 30, 34, 56, 60]. However, we argue that this method is

neither effective nor scalable for long-duration, high-frequency sensor data. To illustrate, consider using HARGPT [28] with 40Hz IMU data. The prompt would need to encode time-series measurements like the following:

Instruction: You are an expert of IMU-based human activity analysis.
Question: The IMU data is collected from smartphone and smartwatch with a sampling rate of 40Hz. The IMU data is given in the IMU coordinate frame. The three-axis accelerations and gyroscopes are given below.
 Accelerations:
 x-axis: 0.34979248, 0.34767151, ..., y-axis: -0.42715454, -0.42863464, ..., z-axis: -0.89616394, ...
 Gyroscope:
 x-axis: -0.11219972, -0.20297755, ..., y-axis: -0.08648433, -0.20838715, ..., z-axis: 0.03115442, ...
 Please answer the following question based on the given information and IMU readings: {Question}.
 Please make an analysis step by step.
Response:

While this type of input is manageable for short durations (e.g., 10 seconds generating 2,400 values), it becomes infeasible for longer periods. For example, 24 hours of 6-axis IMU data at 40Hz would generate approximately 20 million values—equivalent to roughly 50 million tokens (assuming 10 characters per number and 1 token per 4 characters). This significantly exceeds the context limits of even the most advanced LLMs, such as GPT-4 (8K tokens) [5] or GPT-4o (128K tokens) [5]. In addition to input size limitations, LLMs are known to struggle with long-context reasoning [20, 25], making it difficult to extract accurate answers from such lengthy inputs. While recent work has proposed extracting low-frequency features (e.g., activity labels, summary statistics) and converting them into text logs [57], it remains an open question which features best preserve meaningful information across diverse user queries, while remaining within the token limits of current LLMs. For these reasons, we did not include baselines from this category. In contrast, SensorChat addresses these challenges by encoding long-duration raw sensor data into compact embeddings and incorporating a dedicated query stage, enabling scalable and accurate reasoning across extended time periods.

- (2) Visualizing raw sensor signals as images and feeding them into LLMs is another approach that may help models understand sensor data [62]. However, the complexity and scale of the data in our scenario introduce challenges that typical visualization approaches are not well-equipped to address. Visualizing long-duration, high-frequency data often leads to dense and cluttered images, making it difficult to interpret or extract actionable insights. For instance, as shown in Fig. 10, even a simple plot of the first axis of smartphone IMU data for a single user over multiple days results in a figure that is hard to parse, especially when attempting to answer concrete time-related questions. Such time queries seek precise minute-level answers, which are common in QA benchmarks like SensorQA [47]. As such, we did not include direct comparisons with baselines in this category. While adaptively selecting relevant segments for visualization based on user queries is a potential direction, it remains an open challenge to determine appropriate time windows automatically. SensorChat avoids this issue by incorporating an explicit query stage to efficiently retrieve and reason over relevant sensor data.
- (3) Multimodal LLMs use a pretrained sensor encoder or adapter to compress raw sensor measurements into embeddings, which are then passed to an LLM [11, 21, 37, 38]. In our evaluation (Sec. 6), methods like LLaMA-Adapter [63], LLaVA-1.5 [33], and OneLLM [21] fall into this category. These approaches perform well on tasks like video captioning or QA on short IMU sequences, but struggle to scale when applied

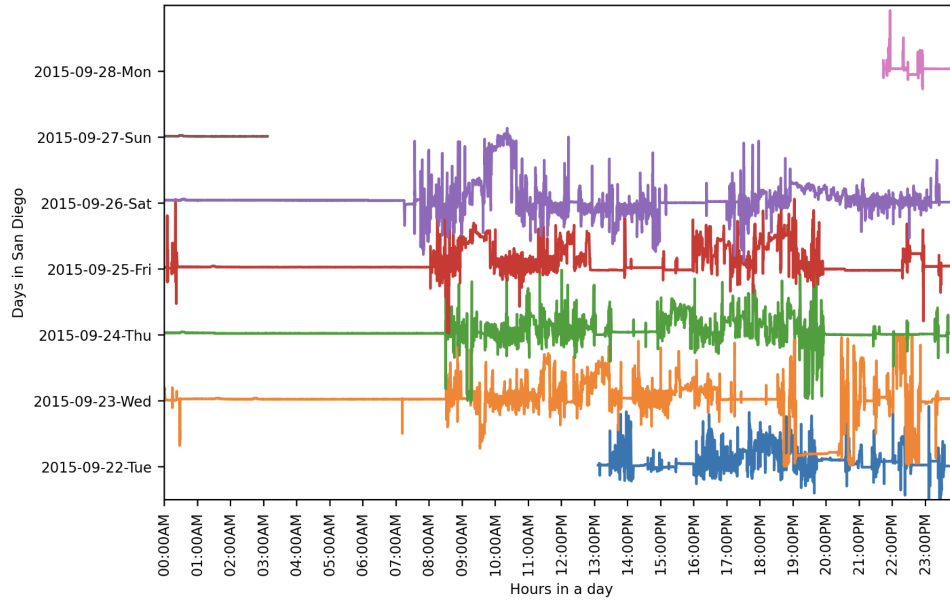


Fig. 10. Visualization of the first axis of smartphone IMU signals. These visualizations pose extreme challenges in answering both quantitative and qualitative questions, such as “How long was I at school?” or “Did I go to school?”

to long-duration, high-frequency sensor data. For example, OneLLM converts every 10-second chunk of IMU data into a 512-dimensional embedding. Over a full day, this results in an embedding matrix of 8,640 rows and 512 columns. If each element were treated as a token, the total input size would reach 4.4 million tokens. Similar as case (1), such long input token sequence far exceeds the token limits of state-of-the-art LLMs and likely leads to inaccurate responses.

- (4) The last category of methods use neural network to associate the raw sensor signals with specific context labels, which are then concatenated into a narrative text to LLMs [9, 39, 42]. In our evaluation (Sec.6), IMU2CLIP+GPT4o [39] and LLMsense [42] are the latest contributions belonging to this category. These methods are well-suited to our setting, as they avoid passing long-duration sensor data directly to LLMs, addressing the interpretability and token limit issues seen in earlier approaches. Instead, they use LLMs for reasoning based on pre-processed text summaries. However, both IMU2CLIP+GPT-4o and LLMsense use a passive summarization approach similar to journaling systems [6, 49], by simply stitching together all timestamped activity descriptions into one long narrative. For example, the following shows the start of a typical narrative text generated by IMU2CLIP+GPT4o [39]:

2015-09-30-Wed 13:17PM The person is at main workplace
 2015-09-30-Wed 13:19PM The person is walking and talking at school and at main workplace while the phone is in hand
 2015-09-30-Wed 13:20PM The person is sitting and doing computer work at school and at main workplace

Such long, passively generated narrative text often includes irrelevant information and can exceed LLM token limits, leading to truncated inputs and missed details. To overcome this, SensorChat introduces a dedicated query stage that extracts only the sensor information relevant to the user’s question. This results

in cleaner, more focused inputs for the final answer assembly stage, improving both clarity and accuracy of SensorChat's answers.

In summary, SensorChat adopts the strategy of (4) that is the most suitable for our scenario, that is, QA over long-duration, high-frequency raw sensor measurements. SensorChat further improves on the latest works by introducing a dedicated query mechanism to effectively process long-duration, high-frequency raw sensor signals while fusing knowledge with user questions.

B Custom Query Function Design in SensorChat

In SensorChat, we design the following query functions to handle various real-life QA scenarios.

- `CalculateDuration` is used for time comparisons, time queries, and existence questions.
- `DetectActivity` handles activity queries. `CountingFrequency` and `CountingDays` handle single- and multi-day counting.
- `DetectFirstTime` and `DetectLastTime` are used for specific timestamp queries.

We emphasize that the function set can be expanded in the future to accommodate a broader range of queries. However, since the query function is selected by the LLM during question decomposition, it is crucial to limit the number of functions to avoid overwhelming or confusing the model. Fortunately, one strength of SensorChat is its ability to transform questions into existing functions during question decomposition, eliminating the need to create new ones unnecessarily. For instance, a question like "What did I do right after waking up?" can be decomposed into calls to `DetectFirstTime` and `DetectLastTime`, rather than requiring the creation of a new function such as `DetectNextActivity`. To enable this transformation, we include a list of available functions in the prompt, as shown in Fig. 4a. This approach ensures that the current set of six query functions is sufficient to effectively handle the vast majority of user queries.

C SensorChat Implementation Details

The algorithm part of SensorChat is implemented with Python and PyTorch [44]. For the offline sensor encoder, SensorChat uses a Transformer architecture [53] with 6 encoder layers, 8 attention heads, and a feedforward network size of 2048 for time series data. For the low-frequency phone status data, SensorChat employs a fully connected layer as the encoder. The fusion layer is also implemented as a fully connected layer. The label encoder is initialized from the pretrained CLIP ViT-B/32 label encoder [45]. Both the sensor and label embedding spaces share a dimension size of 512. Offline pretraining is performed with the partial-context loss proposed in Sec. 4.3.1 using a temperature scalar of $\tau = 0.1$. We use the Adam optimizer with a learning rate of $1e - 5$ over 100 epochs.

For question decomposition, we design two solution templates for each question category. Limiting the number of templates to two helps balance prompt length and performance. For online data queries, we initialize the classifier f as a multilayer perceptron with one hidden layer and a ReLU activation function. The hidden layer size is set to 512 and the query threshold is configured to $h = 0.5$. For finetuning the LLM in answer assembly, we apply low rank adaptation (LoRA) finetuning [24] on an NVIDIA A100 GPU, using a batch size of 8, a learning rate of 0.0002 and 10 epochs. The LoRA rank r is 16, the LoRA scaling factor α is 16, and the LoRA dropout is 0.1. During answer generation, SensorChat uses a max sequence length of 1024 and a generating temperature of 0.2.

D Baselines Implementation Details

We consider both closed-source and open-source baselines for a comprehensive analysis, including

- **GPT-4** [5], **T5** [46] and **LLaMA** [50] are popular **text-only** language models that only take questions as input.

- **GPT-4-Turbo [5] and GPT-4o [5]** are **vision+text** baselines taking the activity graphs and the questions as inputs. For all **vision+text** baselines, we feed the activity graphs in SensorQA [47], similar to Fig. ??, along with the questions into the model.
- **LLaMA-Adapter [63]** is a recent **vision+text** framework offering a lightweight method for fine-tuning instruction-following and multimodal LLaMA models. It integrates vision inputs (i.e., activity graphs from SensorQA [47]) with LLMs using a transformer adapter. We utilize the latest LLaMA-Adapter V2 model.
- **LLaVA-1.5 [33]** represents state-of-the-art **vision+text** model. LLaVA connects pre-trained CLIP ViT-L/14 visual encoder [45] and large language model Vicuna [12], using a projection matrix. LLaVA-1.5 [33] achieves state-of-the-art performance on 11 benchmarks through simple modifications to the original LLaVA and the use of more extensive public datasets for finetuning.
- **IMU2CLIP+GPT-4o [39]** is the state-of-the-art **sensor+text** method using IMU2CLIP [39] to associate raw IMU signals with text and using GPT-4o for answer generation. For each timestamp, IMU2CLIP searches for a text description that best matches the IMU data. The complete narrative text from all timestamps is then fed into GPT-4o [5] with the question to generate an answer.
- **LLMSense [42]** is the state-of-the-art model for high-level reasoning over long-term sensor data. LLMSense fuses **sensor+text** by converting neural network classification outputs into text, which forms a list of narrative text for all timestamps. This text is then fed to GPT-4o [5] for reasoning.
- **DeepSQA [55]** trains a CNN-LSTM model with compositional attention to fuse **sensor+text** modalities and predict from a fixed and limited set of candidate answers given questions and IMU signals. We adapt their implementation to use the full-history timeseries data as input, to align with SensorChat's setup.
- **OneLLM [21]** is a state-of-the-art multimodal LLM framework that processes **sensor+text** modalities using a universal pretrained CLIP encoder and a mixture of projection experts for modality alignment. We adapt their implementation and feed the full-history timeseries data from the IMU tokenizer.

We did not compare with the latest works of Sensor2Text [11], PrISM-Q&A [9], and DrHouse [60] due to limited access to their open-source code and models. Table 6 summarizes key implementation details for all methods evaluated on the SensorQA dataset [47]. In particular, we highlight important aspects of input data format and training methods.

Input Format We employ a wide range of state-of-the-art baselines that are designed for various modalities, including text, images or sensors. To ensure a fair comparison with SensorChat, we adapt the long-duration, high-frequency sensor data to align with the original modality format of each baseline. The detailed inputs for each baseline (covering text, image, and sensor modalities) are shown in Table 6.

- For text-only models, i.e., **GPT-4 [5]**, **T5-Base [46]**, **LLaMA2-7B [50]**, we only feed the textual question into the model.
- For models that allow image+text input, i.e., **GPT-4-Turbo [5]**, **GPT-4o [5]**, **LLaMA-Adapter [63]**, **LLaVA-1.5 [33]**, we feed the activity graphs from SensorQA [47] as the image input and the textual question as the text input. We did not adopt the approach of directly visualizing raw sensor signals in graphs and feeding them to LLMs [62]. This is because such visualization of long-duration (more than one day), high-frequency sensor signals (such as those from IMU sensors) are hard to interpret and extract useful insights, as discussed in Appendix A. Instead, the activity graphs in SensorQA clearly illustrate a user's schedule over a multi-day period, making them more suitable for our QA scenario. Note that these baselines use oracle activity graphs as input, which include ground-truth activity information. However, even with access to ground-truth data, their QA performance remains significantly lower than that of SensorChat, which relies on a predictive model.
- Both **IMU2CLIP+GPT-4o [39]** and **LLMSense [42]** rely on a pretrained sensor encoder to generate textual descriptions for each timestamp. The full sequence of narrative text is then fed into a LLM to

Table 6. Comparison of baseline and SensorChat implementation. The baselines are adapted in various ways to handle the long-duration, high-frequency raw sensor data in our setting. The detailed input formats are adjusted based on their original input modality. For all methods, the expected output is a textual answer.

Method	Text Input	Image Input	Sensor Input	Training Method
GPT-4 [5]	Question	-	-	Few-shot learning
T5-Base [46]	Question	-	-	Full training
LLaMA2-7B [50]	Question	-	-	LoRA finetuning [24]
GPT-4-Turbo [5]	Question	SensorQA activity graphs	-	Few-shot learning
GPT-4o [5]	Question	SensorQA activity graphs	-	Few-shot learning
LLaMA-Adapter [63]	Question	SensorQA activity graphs	-	LoRA finetuning [24]
LLaVA-1.5 [33]	Question	SensorQA activity graphs	-	LoRA finetuning [24]
IMU2CLIP+GPT4o [39]	Question	-	Raw IMU signals	First train a neural network using IMU2CLIP [39]. Then, apply few-shot learning on GPT4o [5] for reasoning on narrative text
LLMSense [42]	Question	-	Raw multimodal sensor signals	First, train a neural network for context classification using cross-entropy loss. Then, apply few-shot learning with GPT-4o [5] for reasoning on narrative text
DeepSQA [55]	Question	-	Raw IMU signals	Full training
OneLLM [21]	Question	-	Raw IMU signals	LoRA finetuning [24]
SensorChat (This work)	Question	-	Raw multimodal sensor signals	Use few-shot Learning in the question decomposition stage, and LoRA finetuning [24] in the answer assembly stage.

perform high-level reasoning and produce the final answer. In IMU2CLIP+GPT-4o, we train the sensor encoder using the IMU2CLIP approach [39]. The sensor encoder maps raw IMU signals into an embedding space and retrieves the most similar text embedding. For LLMSense, we train a neural network with the same architecture as the sensor encoder in SensorChat, using cross-entropy loss for context classification. The text description is constructed by combining all predicted positive context labels into a complete sentence, following a set of fixed language rules. Both baselines use GPT-4o [5] as the LLM backbone. We choose GPT-4o for its significantly larger context window (128K tokens vs. 8,192 in GPT-4) and better cost efficiency (compared to GPT-4-Turbo).

- **DeepSQA [55]** and **OneLLM [21]** are multimodal models that accept sensor+text input. We emphasize that both models take the complete history of raw IMU signals associated with a single user as input. DeepSQA uses separate neural networks to process raw IMU signals and textual questions respectively. OneLLM employs an IMU tokenizer to directly process raw IMU signals, which are then passed through a sequence of pretrained universal encoders and a trainable Mixture-of-Experts architecture. The resulting encoded sensor embeddings, along with the text prompt containing the question, are then input into LLaMA2-7B.

For all baselines, we adopt the same hyperparameters and pretrained weights as released in their official codebases.

Training Method As shown in Table 6, all closed-source GPT-based models use a few-shot learning approach. This includes **GPT-4 [5]**, **GPT-4-Turbo [5]**, **GPT-4o [5]**, **IMU2CLIP+GPT4o [39]**, and **LLMSense [42]**. We evaluate both zero-shot and few-shot learning for these baselines, and find that the few-shot approach yields better answer quality and accuracy. To determine the number of examples to include in the prompts, we perform a grid search over 2, 5, 10, 15 on a separate validation set and select 10 examples, which provide the best results. The rest open-source models use full training or LoRA finetuning [24]. More specifically, **T5-Base [46]** and

DeepSQA [55] are fully trained. **LLaMA2-7B** [50], **LLaMA-Adapter** [63], **LLaVA-1.5** [33], **OneLLM** [21] are finetuned with LoRA [24].

SensorChat vs. Baselines We visualize SensorChat’s setup in Table 6 for a clear comparison. Designed for long-term multimodal sensor interactions, SensorChat uses the user question and the full history of raw multimodal sensor signals as input. Unlike LLMsense [42], which converts classification results into a long narrative text and feeds it to an LLM for reasoning, SensorChat performs an explicit sensor data query in the second stage and passes only the relevant query results to the final LLM stage for answer generation. These query results are much shorter, with typically just a few sentences. As a result, SensorChat is more likely to generate accurate answers in the end. SensorChat’s training procedure consists of two LLM-based stages, each employing a different training strategy based on the availability of high-quality datasets. The first question decomposition stage uses few-shot learning due to the lack of a high-quality dataset for this given task. The last answer assembly stage uses LoRA fine-tuning to fully leverage the SensorQA dataset [47], allowing the model to produce answers that are both accurate and aligned with user-preferred styles.

E Additional QA Examples

We illustrate a qualitative comparison of SensorChat with the top-performing baselines in Fig. 11. Specifically, Fig. 11 (a) focuses on time-related queries and (b) focuses on counting questions, which are the most challenging for SOTA methods given the long-duration time series sensor inputs (see Sec. 3.2). Key phrases in the answers are highlighted in the green if they closely match the true answer, and in the red if they do not. The two presented examples are very challenging for state-of-the-art baselines. SensorChat, on the other hand, consistently produces more accurate answers which can be attributed to its novel three-stage pipeline.

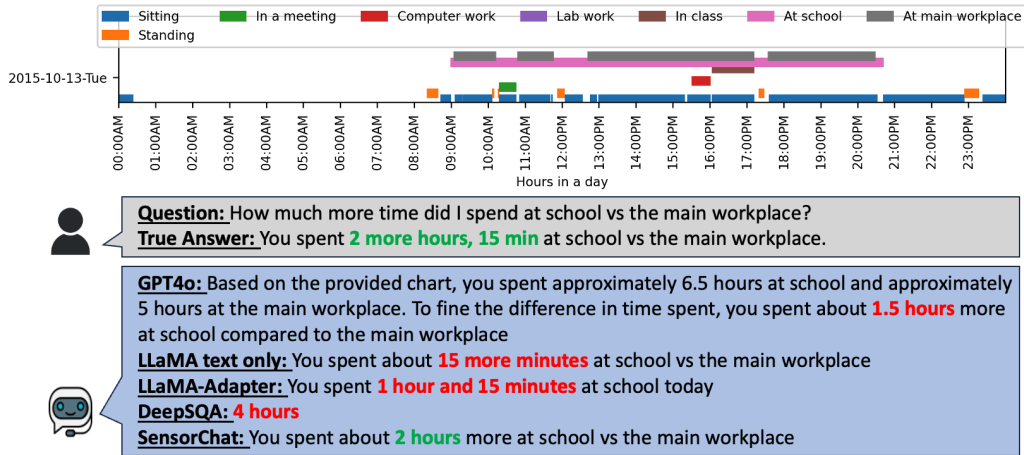
Answering the question in Fig. 11 (a) requires two major steps: (i) calculating the total time spent in school and in the main workplace, and (ii) computing the difference between them. SensorChat accurately answers it by first decomposing and then querying the durations for “at school” and “in the main workplace” respectively, resulting in “*You spent 11 hours and 27 minutes at school Tuesday. You spent 9 hours and 3 minutes at main workplace*”. Finally, SensorChat integrates the above text and determines the time difference during the answering stage. Although SensorChat’s answer is 15 minutes off from the true value, possibly due to inaccuracies in the sensor encoder or LLM reasoning, SensorChat still approximates the ground truth with an accuracy unmatched by other baselines.

In Fig. 11 (b), counting the total days spent at home requires long-term reasoning where LLaMA-Adapter [63] and DeepSQA [55] usually fall short. The performance of IMU2CLIP+GPT-4o [39] is constrained by GPT-4o’s token limit. When the narrative log generated from raw sensor data exceeds this limit, the overflow is discarded, leading to errors in reasoning. In this example, GPT-4o [5] is the only baseline that correctly answers the count-based question by leveraging the activity graph, which clearly indicates the user’s presence at home each day and allows for simple counting. However, GPT-4o may struggle with more complex questions, such as those requiring precise duration calculations. In contrast, SensorChat decomposes this question and queries the duration of “at home” on “each day”, then leaving the counting task to the answering stage. The collaboration across the three stages allows SensorChat to effectively manage a wide range of tasks, particularly those requiring multi-step reasoning and quantitative analyzes, which highlights SensorChat’s advancements over existing works.

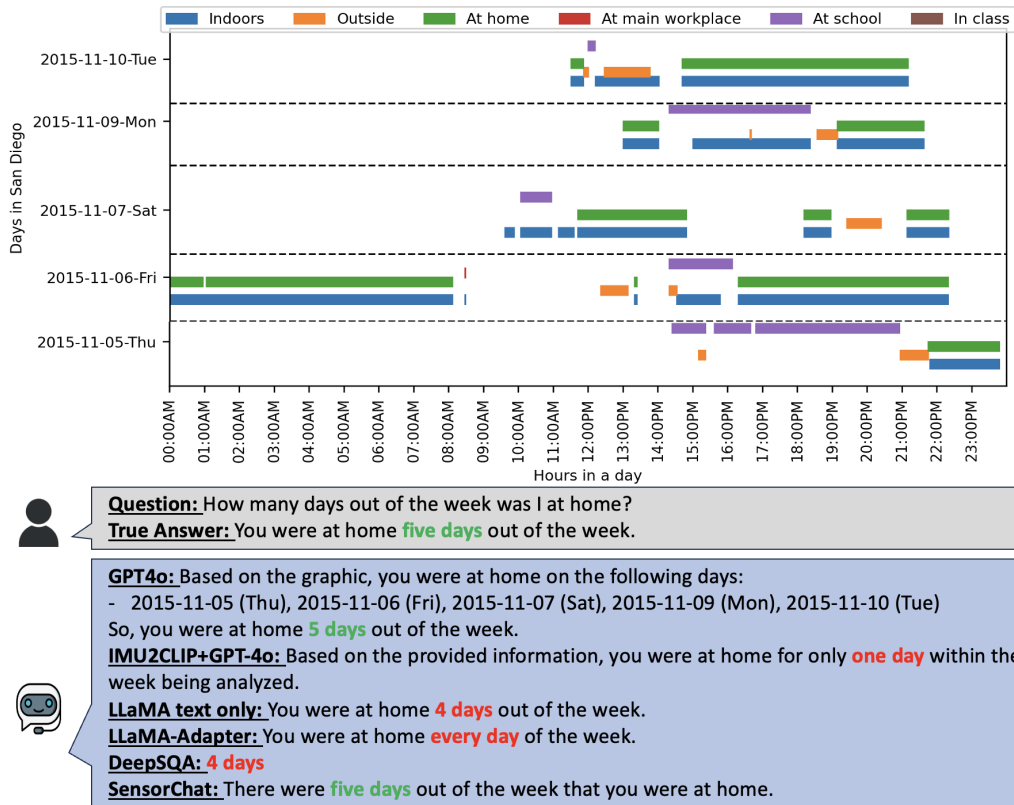
F Ablation Studies

In this section, we comprehensively examine the impact of key design choices in SensorChat. Without loss of generality, we use SensorChat_C as the base model.

Impact of Each Stage in SensorChat We first evaluate the individual contribution of each stage in SensorChat by removing one of them from the pipeline. By removing question decomposition, we use a fixed and general decomposition templates for all questions. Removing sensor data query reverts the model to a language-only



(a) Example of a time query question on a single-day query.



(b) Example of a counting question on a multi-day query

Fig. 11. More qualitative results of SensorChat in comparison to state-of-the-art methods. SensorChat delivers accurate answers for both single-day and multi-day queries, even for complex questions that require multi-step reasoning.

approach. By removing answer assembly, we directly output the queried sensor context from the second stage.

Table 7. Impact of three major stage in SensorChat_C. Bold values highlight the best results. The full SensorChat with all three stages achieve the best results, highlighting the indispensable contribution from each stage.

Setup	Full Answers			Short Answers
	Rouge-1 (↑)	Rouge-2 (↑)	Rouge-L (↑)	Accuracy (↑)
w/o Question Decomposition	0.73	0.57	0.71	0.35
w/o Sensor Data Query	0.72	0.62	0.72	0.26
w/o Answer Assembly	0.26	0.08	0.24	0.0
Full SensorChat	0.77	0.62	0.75	0.54

Table 8. Impact of various designs for sensor-text pretraining in SensorChat_C. Bold values highlight the best results.

Sensor Data	Training Loss	Online Querying Accuracy (↑)	Multiple Choices Answer Accuracy (↑)
Statistical features	Partial-Context Loss	0.91	0.62
Time series	IMU2CLIP [39]	0.90	0.61
Time series	Partial-Context Loss	0.98	0.70

Table 7 summarizes the results on full and short answers including both quality and quantitative accuracy. As observed, removing any stage leads to a significant performance drop. In SensorChat, all three stages must work collaboratively to deliver high-quality, accurate answers across diverse question types in SensorQA. Among the three stages, the answer assembly stage has the most significant impact, as it directly influences the final output. Removing it results in severely degraded performance, with near-zero accuracy on short answers. However, the question decomposition and sensor data query stages are equally crucial.

Impact of Sensor Features and Pretraining Loss Functions We next evaluate the impact of sensor features and loss functions during offline encoder pretraining. Specifically, we compare using statistical features (e.g., mean acceleration) versus raw time series inputs, and IMU2CLIP loss [39] versus our proposed contrastive sensor-text pretraining loss for partial context (see Sec. 4.3.1). For IMU2CLIP, text samples are generated by combining all valid labels into one sentence. We report the online querying accuracy and multiple-choice answer accuracy to assess the influence to sensor information extraction. As shown in Table 8, statistical features result in low querying and answer accuracies. This validates our motivation to design SensorChat that high-frequency time series sensor data are critical for fine-grained activity information. IMU2CLIP training, even with fine-grained data, yields poorer querying and answering accuracies, highlighting its limited ability associating sensor embeddings from partial text query. Our proposed loss function, which aligns sensor and text encoders for partial context queries, proves more effective. These findings emphasize the importance of selecting appropriate sensor features and loss functions during pretraining in order to achieve high-performance QA.

Impact of LLM Design Choices We finally evaluate the impact of various design choices for LLMs. In question decomposition, we assess the contribution of in-context learning (ICL), chain-of-thought (CoT) techniques, and different backbone LLMs. The results are summarized in Table 9. Both ICL and CoT are crucial for high-quality and accurate answers. This is because an effective question decomposition improves sensor data queries. The solution templates in ICL are more essential to SensorChat as removing ICL reduces answer accuracy by 11%. CoT enhances reasoning and slightly boosts accuracy by 1-5%. Interestingly, using a more advanced backbone (GPT-4 vs. GPT-3.5-Turbo) results in minimal improvement in answer quality, as GPT-4, while generating richer text, does not follow instructions as well as GPT-3.5-Turbo according to our observation.

For answer assembly, we evaluate the effectiveness of finetuning compared to zero-shot or few-shot learning, as well as different LLaMA backbones. As shown in Table 9, zero-shot learning results in poor performance, while few-shot learning improves answer quality but still lags behind finetuning. This highlights that finetuning is the

Table 9. Impact of various design choices for LLMs in SensorChat_C. Bold values highlight the best results. The results highlight the necessity of in-context learning (ICL) and chain-of-thought (CoT) during question decomposition, and using finetuning instead of zero- or few-shot learning during answer assembly.

Stage	Setup	Model in Stage	Full Answers			Short Answer
			Rouge-1 (↑)	Rouge-2 (↑)	Rouge-L (↑)	Accuracy (↑)
Question Decomposition	w/o ICL	GPT-3.5-Turbo	0.75	0.59	0.72	0.43
	w/o CoT	GPT-3.5-Turbo	0.76	0.61	0.74	0.50
	Full	GPT-4	0.77	0.62	0.75	0.49
	Full	GPT-3.5-Turbo	0.77	0.62	0.75	0.54
Answer Assembly	Zero-shot learning	LLaMA2-7B	0.16	0.07	0.14	0.0
	Few-shot learning	LLaMA2-7B	0.43	0.29	0.41	0.24
	Finetuning	LLaMA3-8B	0.76	0.61	0.74	0.53
	Finetuning	LLaMA2-7B	0.77	0.62	0.75	0.54

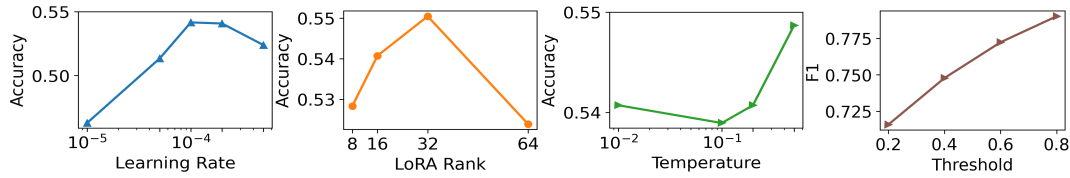


Fig. 12. Sensitivity of key hyperparameters including learning rate (leftmost), rank in LoRA finetuning (middle left), LLM generating temperature (middle right) and query threshold in SensorChat (right most).

most effective approach when a dataset like SensorQA is available. Using a more advanced LLaMA backbone, such as LLaMA3-8B, has minimal impact. Finetuning and the dataset prove to be more important than the model architecture during answer assembly.

G Sensitivity Analyses

Fig. 12 shows the sensitivity of key parameters in SensorChat, while the less sensitive ones are omitted due to space limitation. The default parameter setting is the same as described in Sec. 5. We mainly focus on evaluating the short answers to assess the parameters' impact on factual information extraction.

Learning Rate in Answer Assembly Fig. 12 (leftmost) shows the short answers accuracy for learning rates of $\{1e-5, 5e-5, 1e-4, 2e-4, 5e-4\}$ during finetuning. Larger learning rates result in bigger gradient steps during LoRA finetuning, with $1e-4$ providing the best performance for our task.

LoRA Rank in Answer Assembly Fig. 12 (middle left) shows the short answers accuracy for LoRA ranks of $\{8, 16, 32, 64\}$ during finetuning. The rank affects the size of the LoRA adapter weights. Higher ranks mean more parameters and a larger weight space to optimize. For our task, varying ranks have little impact on final accuracy, with rank 32 achieving the best performance for short answers.

Generating Temperature in Answer Assembly Fig. 12 (middle right) shows the short answers accuracy for generating temperatures of $\{0.01, 0.1, 0.2, 0.5\}$ during the final answer generation. Higher temperatures instruct the LLM to use more “creativity”. For our task, varying temperatures have negligible impact on the short answers accuracy, indicating minimal impact to the sensor information extraction.

Query Threshold in Sensor Data Query Fig. 12 (rightmost) shows the F1 scores of online querying at different query thresholds $h = \{0.2, 0.4, 0.5, 0.6, 0.8\}$. Raising the threshold h excludes less confident positive predictions, which can improve F1 scores. However, this may also overlook some detailed events, potentially reducing answer accuracy. Ideally, h should be calibrated individually for each user to achieve the best results.

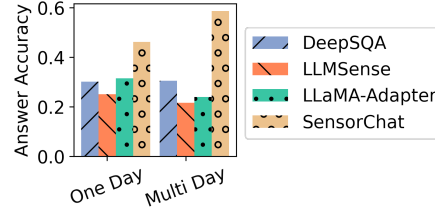


Fig. 13. Performance of SensorChat compared to representative baselines on SensorQA [47], categorized by the duration of sensor data associated with each user question.

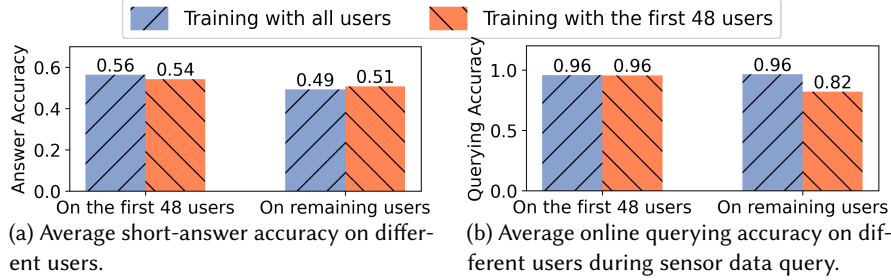


Fig. 14. Generalization performance of SensorChat’s answer assembly stage (a) and online query stage (b) when trained on data from different users. Results show that the answer assembly stage generalizes well across users, while the sensor data query stage is more sensitive to shifts in user data distribution.

H Impact of Sensor Data Duration

Fig. 13 presents the performance of SensorChat and representative baselines across different sensor data durations using SensorQA [47]. We categorize user questions based on the associated sensor data into single-day and multi-day groups and report short answer accuracy. The baselines include DeepSQA [55], LLMsense [42], and LLaMA-Adapter [63], each representing a mainstream approach. As discussed in Sec. 6.3, these methods struggle with long-duration, high-frequency sensor data. Fig. 13 further shows that their performance declines when scaling from single-day to multi-day inputs: LLaMA-Adapter drops from 0.31 to 0.23, and LLMsense from 0.25 to 0.21. This highlights their limited ability to reason over long-term sensor data. Specifically, LLaMA-Adapter compresses sensor signals into embeddings before passing them to the LLM, which becomes less effective as the time span grows. LLMsense’s drop in performance stems from LLMs’ known difficulty with long-context reasoning [20, 25], often resulting in confusion and inaccurate responses. In contrast, SensorChat achieves higher accuracy on multi-day queries (0.58) than on single-day ones (0.46), demonstrating its strength in handling long-duration questions. However, this does not mean SensorChat underperforms on short durations. The slightly lower single-day accuracy is due to the nature of those queries, which demand precise time-based answers more frequently than multi-day queries, such as, “How long did I exercise today?” As discussed in Sec. 6.3, short answer accuracy is a strict metric for such questions, leading to lower scores compared to multi-day queries (e.g., “On which day did I exercise?”).

I Generalization and Robustness

We evaluate SensorChat’s generalization and robustness across different users’ sensor data and QA inputs. In addition to the standard 80/20 random split, we compare results with a split where training is performed on the first 48 out of 60 users and testing includes all users. To ensure a fair comparison, we equalize the training set size in both splits by duplicating samples in the smaller set. Our evaluation focuses on two key learning processes in SensorChat: LLM fine-tuning in answer assembly and sensor-text encoder pretraining.

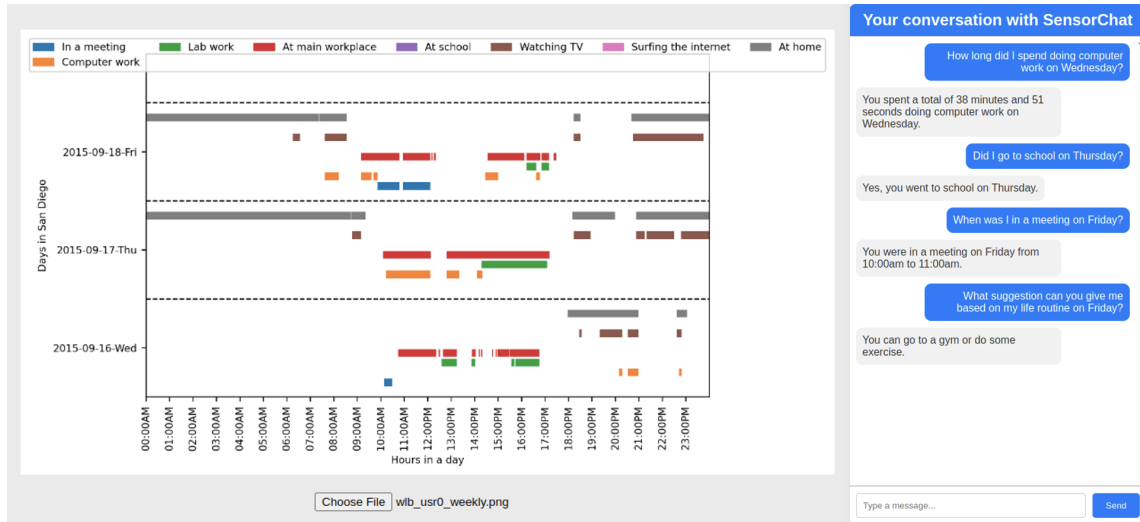


Fig. 15. A prototype Graphical User Interface (GUI) for SensorChat. The left panel displays an optional activity visualization based on model predictions from raw sensor data. We plan to enhance interactivity by allowing users to select specific days and context labels of interest in the future. The right panel shows the main conversation between users and the SensorChat system: user questions appear in blue box on the right, and SensorChat responses appear in gray box on the left.

Fig. 14a presents the short-answer accuracy when fine-tuning on all users' QA data versus only the first 48 users. The results show similar accuracy for both seen and unseen users, demonstrating SensorChat's strong generalizability in answer assembly. This is likely due to SensorChat's design of treating answer assembly as a pure language task. Since all users' language tokens follow similar distributions in a sensor-based QA task, generalization remains robust across user variations.

Fig. 14b presents the online querying accuracy when pretraining with all users' sensor data versus only the first 48 users. Unlike language fine-tuning, limiting sensor data to the first 48 users leads to accuracy degradation on unseen users due to variations in data distributions. Therefore, improving SensorChat's generalization to new users primarily depends on developing a robust sensor and text encoder, which is a well-studied problem in existing literature [58]. We leave the investigation for combining with these techniques in future work.

Generalizability Across Sensor Device Placements We acknowledge that sensor data from vastly different body positions could introduce ambiguity in the encoder. However, detailed device placement information may not always be available, depending on the sensor setup. In this work, SensorChat adopts a generalized approach without assuming prior knowledge of sensor placement. The key idea that SensorChat could work in this case is that, by training on a large-scale dataset collected from diverse users with varying device placements, the model learns a general similarity association between sensor data and context labels. When the user queries about a specific context of their lives, SensorChat could retrieve all sensor recordings sufficiently similar using the learned similarities. Additionally, if device placement information is available, we foresee that integrating SensorChat with recent sensor context augmentation techniques, such as [14], could further enhance performance. We leave this as an avenue for future exploration.

J Graphical User Interface of SensorChat

Figure 15 presents a prototype of the SensorChat GUI used in our user study. Users primarily interact with SensorChat by posing questions and receiving responses, as illustrated in the right panel of the figure. These questions may be either quantitative or qualitative. In this example, the system operates with a low temperature

setting for answer generation (see Appendix G for additional sensitivity experiments), resulting in concise responses that emphasize key facts. This temperature parameter can be adjusted to better align with individual user preferences. The left panel of the GUI includes an optional activity visualizer that provides users with an overview of their schedule. This visualizer can help remind users of their schedule and guide them toward more insightful questions. For lightweight deployments, such as the SensorChat_E mobile app on edge devices, this visualizer can be disabled, retaining only the conversation panel for streamlined interaction.

We emphasize that this interface represents an initial prototype of SensorChat. Future enhancements will focus on improving interactivity, such as enabling users to select specific dates and context labels for visualization in the left panel. Additionally, we plan to incorporate a raw sensor data viewer for users interested in inspecting signals captured from their mobile or wearable devices.

K Questions Asked in The Real User Study

The goal of the real user study is to evaluate SensorChat's generalizability and practicality in real-world scenarios, extending beyond the predefined scope of SensorQA [47]. We notice that some participant questions overlap with those in SensorQA, particularly when they are quantitative. The participants also raised a variety of qualitative questions that reflect their personal interests and are not included in SensorQA. After interacting with SensorChat and posing a mix of both qualitative and quantitative questions, participants reported fairly high satisfaction, demonstrating SensorChat's adaptability and usefulness in real-life scenarios.

For example, the following quantitative questions are raised by the participants, which are similar to those in SensorQA:

How much time did I spend sitting down vs standing up?
 How much time did I spend doing lab work?
 How much time on average did I spend on entertainment for the past 2 days?
 Did I perform any household chores, if so, what chores did I do?
 What did I do after having lunch?

The following qualitative questions are posed by the participants during real user study, which are **not** present in SensorQA:

What do you suggest to make me more active?
 How would you rate this lifestyle and what do you suggest that I improve on?
 Did I exercise enough for a young adult?
 Do I like cycling?
 Am I attentive when talking to my co-workers?